Quantum Error Correction

Lecture Notes — Version 3.0

Extended Edition with Worked Examples and Visual Aids

Quantum Computing Course

May 13, 2026

Abstract

These lecture notes provide a comprehensive, self-contained introduction to quantum error correction (QEC) aimed at undergraduate students with basic linear algebra and introductory quantum mechanics. We begin with the question of *why* quantum information is fragile, build from classical intuition to the 3-qubit bit-flip and phase-flip codes, study the Shor 9-qubit code and the Steane $[\![7, 1, 3]\!]$ code, develop the stabilizer formalism as a unifying language, and then move to the topological surface code. Fault-tolerant gates, magic state distillation, the threshold theorem, and advanced quantum LDPC and color codes round out the material.

Contents

1 Introduction: Why Quantum Error Correction?

Intuition

Imagine writing a message on a whiteboard in a room with a randomly blowing fan. Every few seconds a gust might smudge a letter. After 1000 letters the message is unreadable. Classical computers deal with this by building transistors so reliable that the "fan" almost never blows. Quantum computers *cannot* do the same thing: the "fan" is the quantum environment itself, and it can never be fully shut out. Quantum error correction is the strategy of encoding the message so cleverly that even with constant smudging, the original meaning can always be recovered.

Quantum systems are notoriously fragile. Qubits, unlike classical bits, are susceptible to **decoherence** and operational errors from two main sources:

- **Decoherence.** A qubit interacts with its surroundings (stray magnetic fields, vibrations, nearby electrons). This interaction *entangles* the qubit with the environment, causing the delicate quantum superposition to leak away. The characteristic times are called T_1 (energy relaxation) and T_2 (dephasing). For state-of-the-art superconducting qubits, $T_2 \sim 100\,\mu$s; a typical gate takes ~ 50 ns, so only about 2000 gates are possible before significant decoherence.
- **Gate errors.** No physical operation is perfect. A CNOT gate might have a 0.1–1% error probability. For a 1000-gate algorithm on a single unprotected qubit with gate error $p = 0.001$, the probability of *no error at all* is $(1-0.001)^{1000} \approx 0.37$. The chance of getting the correct answer with high probability is essentially zero.

Key Point

The goal of quantum error correction is to **detect and correct errors without measuring (and thus collapsing) the quantum information**. This is fundamentally different from classical error correction, and requires genuinely new ideas.

1.1 A Concrete Failure Rate Calculation

Let us make the problem concrete before developing solutions.

Worked Example 1.1: Failure probability accumulation

Suppose we run an algorithm that requires $n = 1000$ sequential single-qubit gates. Each gate has error probability $p = 0.01$ (1%). What is the probability that at least one gate fails?

Solution. The probability that a *single* gate succeeds is $1 - p = 0.99$. The probability that *all* n gates succeed is:

$$P(\text{no error}) = (1-p)^n = (0.99)^{1000} \approx e^{-10} \approx 4.5 \times 10^{-5}.$$

So with overwhelming probability ($\approx 99.995\%$), at least one gate fails. The computation is useless without error correction.

For comparison, if we could reduce the effective logical error rate to $p_L = 10^{-6}$ using QEC (a reasonable target for near-future devices), the same 1000-gate algorithm succeeds with probability $(1 - 10^{-6})^{1000} \approx 0.999$, which is excellent.

1.2 Why Classical Error Correction is Not Enough

Classical error correction often uses **repetition codes** or parity checks. For example, a classical 3-bit repetition code encodes:

$$0 \longrightarrow 000, \qquad 1 \longrightarrow 111,$$

and recovers from a single bit-flip using majority voting. However, applying this idea directly to qubits fails for three fundamental reasons:

1. **The no-cloning theorem** (§1.3) forbids copying an unknown quantum state. We cannot produce three copies of $|\psi\rangle$ to store redundantly.
2. **Errors are continuous.** A qubit can be "partially rotated" by any angle θ. There is an uncountably infinite number of possible errors, unlike the two classical errors (0 and 1).
3. **Measurement destroys superpositions.** If we try to "look at" a qubit to check for errors, we collapse its state.

1.3 The No-Cloning Theorem

Theorem 1.1 (No-Cloning Theorem)**.** *There is no unitary operation U such that for* all *quantum states $|\psi\rangle$:*

$$U\big(|\psi\rangle \otimes |0\rangle\big) = |\psi\rangle \otimes |\psi\rangle\,.$$

Proof by contradiction. Suppose such a U exists. Let $|\psi\rangle$ and $|\phi\rangle$ be any two single-qubit states. By assumption:

$$U(|\psi\rangle\,|0\rangle) = |\psi\rangle\,|\psi\rangle\,, \qquad U(|\phi\rangle\,|0\rangle) = |\phi\rangle\,|\phi\rangle\,.$$

Since U is unitary it preserves inner products:

$$\langle\psi|\,\langle 0|0\rangle\,|\phi\rangle = \big(\langle\psi| \otimes \langle\psi|\big)\big(|\phi\rangle \otimes |\phi\rangle\big),$$

which simplifies to

$$\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2\,.$$

This forces $\langle\psi|\phi\rangle \in \{0, 1\}$. Since $|\psi\rangle$ and $|\phi\rangle$ were *arbitrary*, this is a contradiction: two distinct non-orthogonal states (e.g. $|0\rangle$ and $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$) have $\langle 0|+\rangle = \frac{1}{\sqrt{2}} \neq 0, 1$. □

Intuition

The no-cloning theorem does *not* say we cannot protect quantum information — it says we cannot protect it by simply making copies. Quantum error correction instead spreads the information across an *entangled* state of multiple qubits. No single qubit holds a copy; the information is holographically encoded in the correlations.

1.4 The Three-Step Strategy of QEC

Every quantum error correcting code works through three steps:

1. **Encode.** Map one logical qubit $\alpha\,|0\rangle + \beta\,|1\rangle$ into an entangled state of n physical qubits. The coefficients α, β are *not* stored in any single qubit; they live in global correlations.
2. **Syndrome measurement.** Use ancilla (helper) qubits to measure certain *parities* of the data qubits. These parity measurements (called **syndromes**) reveal *what* error occurred without revealing *what* α and β are.

3. **Recovery.** Apply a corrective operation based on the syndrome. Because the syndrome uniquely identifies the error (for correctable errors), we can undo the damage and restore the original encoded state exactly.

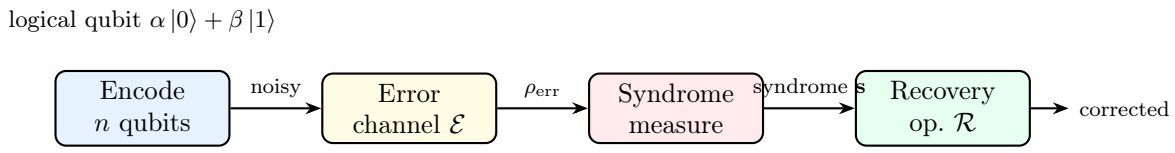


Figure 1: The three-step QEC pipeline. The logical qubit is first encoded into n physical qubits. After an error channel acts, the syndrome is measured (without disturbing α, β), and a recovery operation restores the original state.

1.5 Historical Context

Quantum error correction emerged in the mid-1990s in rapid succession:

- **Shor (1995):** First QEC code — a 9-qubit code correcting arbitrary single-qubit errors.
- **Steane (1996):** 7-qubit CSS code derived from classical Hamming codes; simpler and more efficient.
- **Knill & Laflamme (1997):** General algebraic conditions (the KL conditions) characterising when a set of errors is correctable.
- **Kitaev (1997):** Toric/surface code — topological QEC with local stabilizers, high threshold, and deep connections to topology.
- **Preskill (1997–2012):** Threshold theorem and fault-tolerant computing framework.

1.6 Learning Goals for This Chapter

After reading this chapter, you should be able to:

- Calculate the probability that an unprotected quantum computation fails due to accumulated errors.
- State and prove the no-cloning theorem and explain why it does *not* forbid QEC.
- Describe the encode–syndrome–recover strategy in plain language.
- Explain the three fundamental obstacles preventing direct application of classical error correction to qubits.

Exercise 1.1. A quantum algorithm requires $n = 500$ two-qubit gates, each with error probability $p = 0.005$. What is the probability that no error occurs? What is the minimum gate fidelity required to ensure the computation succeeds with probability at least 0.9?

Exercise 1.2. Prove that the no-cloning theorem forbids *broadcasting*: there is no quantum channel $\mathcal{E}$ such that $\mathcal{E}(|\psi\rangle\langle\psi|) = |\psi\rangle\langle\psi| \otimes |\psi\rangle\langle\psi|$ for all pure states $|\psi\rangle$. (*Hint:* use a linearity argument rather than unitarity.)

Exercise 1.3. Explain in your own words why the no-cloning theorem does *not* prevent quantum error correction. What feature of QEC allows it to protect information without copying it?

Section Summary

Section 1 key points:

- Qubit errors accumulate rapidly; even 0.1% gate error makes a 1000-gate algorithm unreliable without correction.
- No-cloning theorem: we cannot copy an unknown quantum state.
- QEC works by encoding into entangled states, measuring syndromes (error indicators without revealing logical information), and applying corrective operations.

2 Quantum Errors and the Pauli Framework

Intuition

A single qubit lives on the surface of the Bloch sphere: the north pole is $|0\rangle$, the south pole is $|1\rangle$, and every other point is a superposition. An error is a rotation of this sphere. Every rotation decomposes into rotations about three coordinate axes, generated by X (which swaps $|0\rangle \leftrightarrow |1\rangle$), Z (which flips the relative phase), and Y (which combines both effects). This decomposition is the key insight that turns *infinitely many possible errors* into a *finite correctable list* — but to understand *why* it works, we need to build up the picture carefully from physical first principles.

2.1 Single-Qubit Error Types

Consider a qubit in state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$. Although a qubit can undergo infinitely many possible errors (arbitrary rotations or phase changes), it is sufficient to study a small set of fundamental error operators. The Pauli operators $\{I, X, Y, Z\}$ form a **basis** for all 2×2 complex matrices — this means any physical error operator can be expressed as a linear combination of them.

We therefore focus on the following three "pure" error types:

Error	**Op.**	**Action**	**Physical meaning**
Bit-flip	X	$\|0\rangle \leftrightarrow \|1\rangle$	Like a classical bit-flip; exchanges amplitudes
Phase-flip	Z	$\|0\rangle \mapsto \|0\rangle$, $\|1\rangle \mapsto -\|1\rangle$	Flips the relative phase between basis states
Bit+Phase	$Y{=}iXZ$	$\|0\rangle \mapsto i\|1\rangle$, $\|1\rangle \mapsto -i\|0\rangle$	Combines both bit-flip and phase-flip effects

2.2 Pauli Matrices

The four **Pauli operators** are:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

They are Hermitian ($P^\dagger = P$), unitary ($P^\dagger P = I$), and traceless for $P \in \{X, Y, Z\}$. Their key algebraic properties are:

$$X^2 = Y^2 = Z^2 = I, \tag{1}$$

$$XY = iZ, \quad YZ = iX, \quad ZX = iY, \tag{2}$$

$$\{X, Z\} = XZ + ZX = 0 \quad \text{(anticommute)}. \tag{3}$$

The Paulis *commute* with themselves and *anticommute* with each other.

Worked Example 2.1: Verifying Pauli algebra

Let us verify $ZX = iY$ and $XZ = -iY$ directly:

$$ZX = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix},$$

$$iY = i\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix},$$

so $ZX = iY$. Computing XZ gives:

$$XZ = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = -ZX = -iY,$$

confirming both $ZX = iY$ and $XZ = -iY$, consistent with $\{X, Z\} = XZ + ZX = 0$. □

2.3 Discretisation of Quantum Errors

Theorem 2.1 (Discretisation of Errors)**.** *Any single-qubit error E can be written as a linear combination of Pauli operators:*

$$E = c_0 I + c_1 X + c_2 Y + c_3 Z, \qquad c_i \in \mathbb{C}.$$

*Consequently, if a quantum code can correct all single-qubit Pauli errors $\{I, X, Y, Z\}$, it can correct **any** single-qubit error.*

Proof sketch. The four Pauli matrices form a basis for the vector space of 2×2 complex matrices. Any 2×2 matrix E can therefore be so decomposed. When the syndrome is measured, the measurement projects the error onto one of its Pauli components; the remaining component is a pure Pauli error, which the code corrects. □

Intuition

Think of it like this: your code only knows how to handle three "standard" faults (call them X, Y, Z). When some exotic fault E happens, *the act of checking which standard fault occurred* forces the actual error to collapse into one of those standard faults. After the check, you have an ordinary fault you already know how to fix. This is the magic of quantum measurement used constructively.

Worked Example 2.2: Decomposing a rotation error

A common physical error is an **over-rotation**: the qubit rotates by θ about the X-axis instead of staying fixed. The error operator is

$$E(\theta) = e^{-i\theta X/2} = \cos(\theta/2)\, I - i \sin(\theta/2)\, X.$$

For $\theta = 0.1$ rad: $c_0 \approx 0.9988$, $c_1 \approx -0.0500i$. When the syndrome is measured, the error collapses to I (probability ≈ 0.998) or X (probability ≈ 0.0025) — and the code handles both outcomes correctly.

2.4 Quantum Channels and Kraus Operators

2.4.1 Where do Kraus operators come from?

So far we have described errors as single Pauli operators acting on a qubit. But in reality, a qubit does not sit in isolation — it is continuously interacting with its surrounding environment (stray electric fields, phonons, neighbouring electrons). We need a formalism that captures this messy, probabilistic, many-body physics in a compact mathematical form. That formalism is the **quantum channel**, described by Kraus operators.

Intuition

Imagine placing your qubit in a box with a tiny hole. Unknown "signals" leak in and out through the hole, disturbing the qubit in unpredictable ways. You cannot track what goes through the hole. A quantum channel describes exactly how the qubit's state changes when you average over all possible disturbances you *cannot* see.

2.4.2 Derivation of the Kraus Representation

Suppose the qubit starts in state ρ. We use the **density matrix** ρ here (rather than a state vector $|\psi\rangle$) because after tracing out the environment the qubit will generally be in a *mixed* state — a classical probability distribution over quantum states — which density matrices describe and pure state vectors cannot. For a pure initial state, $\rho = |\psi\rangle\langle\psi|$. The environment starts in a known pure state $|e_0\rangle$. Together, the combined system evolves *unitarily* under U (all of quantum mechanics is unitary at the fundamental level):

$$\rho \otimes |e_0\rangle\langle e_0| \xrightarrow{U} U\big(\rho \otimes |e_0\rangle\langle e_0|\big)U^\dagger.$$

But we do not observe the environment. We *trace it out* by summing over all environmental basis states $\{|e_k\rangle\}$:

$$\mathcal{E}(\rho) = \mathrm{Tr}_{\mathrm{env}}\Big[U\big(\rho \otimes |e_0\rangle\langle e_0|\big)U^\dagger\Big] = \sum_k \langle e_k| U\,(\rho \otimes |e_0\rangle\langle e_0|)\,U^\dagger\,|e_k\rangle.$$

Now define the **Kraus operator** for the k-th environmental outcome:

$$\boxed{K_k \;\equiv\; \langle e_k|\,U\,|e_0\rangle.}$$

This is a 2×2 matrix acting on the qubit alone. To see why: U acts on the joint (qubit $\otimes$ environment) space; sandwiching it between environment states $\langle e_k|$ and $|e_0\rangle$ traces away the environment index, leaving an operator purely on the qubit's two-dimensional Hilbert space. With this definition the channel becomes:

$$\boxed{\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger.}$$

Why the completeness condition $\sum_k K_k^\dagger K_k = I$? This follows directly from U being unitary. Unitarity means $U^\dagger U = I_{\mathrm{total}}$, which forces

$$\sum_k K_k^\dagger K_k = \sum_k \langle e_0|\,U^\dagger\,|e_k\rangle\langle e_k|\,U\,|e_0\rangle = \langle e_0|\,U^\dagger U\,|e_0\rangle = \langle e_0|e_0\rangle = I.$$

Physically this completeness condition ensures that probabilities sum to 1: $\mathrm{Tr}[\mathcal{E}(\rho)] = \mathrm{Tr}[\rho] = 1$ for any valid state ρ.

2.4.3 Physical Meaning of Each Kraus Operator

Each K_k describes what happens to the qubit *given* that the environment "jumped" to state $|e_k\rangle$.

- The probability that the environment ends in state $|e_k\rangle$ (and thus that "error k" occurred) is $p_k = \mathrm{Tr}[K_k \rho K_k^\dagger]$.

- The (normalised) post-error state, *if* we somehow knew the environment made this jump, would be $K_k \rho K_k^\dagger / p_k$.
- Since we *do not* know which jump occurred, we add all the outcomes incoherently: $\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger$.

The Kraus representation is thus the mathematical statement:

"Average over all the ways the environment could have disturbed the qubit, weighted by how likely each disturbance is."

Caution

The Kraus decomposition of a channel is **not unique**. Different sets $\{K_k\}$ and $\{\tilde{K}_k\}$ related by a unitary matrix $\tilde{K}_k = \sum_j u_{kj} K_j$ describe the *same* physical channel. Physically, this freedom reflects that we can choose different bases for the environment's Hilbert space. What is unique is the channel $\mathcal{E}$ itself.

2.4.4 Key Examples

Bit-flip channel (probability p). With probability $1-p$ nothing happens; with probability p the qubit is flipped:

$$K_0 = \sqrt{1-p}\, I, \qquad K_1 = \sqrt{p}\, X.$$

Check completeness: $K_0^\dagger K_0 + K_1^\dagger K_1 = (1-p)I + pI = I$. ✓

The channel acts as:

$$\mathcal{E}(\rho) = (1-p)\rho + p\, X\rho X.$$

Dephasing (phase-damping) channel. Only Z-type errors occur with probability p:

$$K_0 = \sqrt{1-p}\, I, \qquad K_1 = \sqrt{p}\, Z.$$

$$\mathcal{E}_{\text{deph}}(\rho) = (1-p)\rho + p\, Z\rho Z.$$

In the Bloch sphere picture, dephasing *contracts the equatorial plane*: the x- and y-components of the Bloch vector shrink while the z-component is unchanged.

Depolarising channel. With probability $1-p$ nothing happens; otherwise a Pauli error X, Y, Z each occurs with probability $p/3$:

$$\mathcal{E}_{\text{dep}}(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z).$$

The Kraus operators are $\{K_0, K_1, K_2, K_3\} = \{\sqrt{1-p}\, I,\ \sqrt{p/3}\, X,\ \sqrt{p/3}\, Y,\ \sqrt{p/3}\, Z\}$.

Amplitude damping channel. This models energy relaxation (T_1 decay): the qubit decays from the excited state $|1\rangle$ to the ground state $|0\rangle$ with probability $\gamma = 1 - e^{-t/T_1}$. The Kraus operators are:

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \qquad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix}.$$

Intuition for K_0 and K_1:

- K_0 is the "no-jump" operator. It leaves $|0\rangle$ unchanged and contracts $|1\rangle$ towards $|0\rangle$ by the factor $\sqrt{1-\gamma}$. This describes the qubit *not* having decayed yet but having lost some of its "potential" to decay.

- K_1 is the "jump" operator. It maps $|1\rangle \to \sqrt{\gamma}\,|0\rangle$ and annihilates $|0\rangle$ (you cannot decay from the ground state). This is the discrete event "the qubit emitted a photon and fell to $|0\rangle$", which occurs with probability γ.

Check completeness:

$$K_0^\dagger K_0 + K_1^\dagger K_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1-\gamma \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & \gamma \end{pmatrix} = I. \checkmark$$

Worked Example 2.3: Depolarising channel on $|+\rangle$

Let $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ so $\rho = |+\rangle\langle+| = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. Apply $\mathcal{E}_{\text{dep}}$ with $p = 0.12$:

$$\begin{aligned} X\rho X &= \rho \quad (|+\rangle \text{ is a } +1 \text{ eigenstate of } X), \\ Y\rho Y &= \tfrac{1}{2}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} = |-\rangle\langle-|\,, \\ Z\rho Z &= |-\rangle\langle-|\,. \end{aligned}$$

Therefore:

$$\mathcal{E}(\rho) = 0.88\,\rho + 0.04\,\rho + 0.04\;|-\rangle\langle-| + 0.04\;|-\rangle\langle-| = 0.92\,\rho + 0.08\;|-\rangle\langle-|\,.$$

The off-diagonal coherences shrink: the channel has partially *dephased* the state.

2.5 Multi-Qubit Pauli Group

The single-qubit picture extends naturally to n qubits, but two important subtleties arise: errors become sums of tensor products (not just single tensor products), and phases $\{\pm 1, \pm i\}$ must be tracked to keep the algebra consistent. This subsection explains both points from first principles.

2.5.1 Why tensor products?

Start with the physical picture. For n qubits, a tensor product

$$P^{(1)} \otimes P^{(2)} \otimes \cdots \otimes P^{(n)}, \qquad P^{(j)} \in \{I, X, Y, Z\},$$

is a **specific error operator**: it says qubit 1 experiences error $P^{(1)}$, qubit 2 experiences error $P^{(2)}$, and so on independently. The superscript (j) is simply the qubit label. For example, $X \otimes I \otimes Z$ means qubit 1 suffers a bit-flip, qubit 2 is untouched, and qubit 3 suffers a phase-flip — all simultaneously and independently.

Now the linear algebra observation. By choosing $P^{(j)}$ from $\{I, X, Y, Z\}$ independently at each of the n qubit positions, we obtain 4^n distinct such operators in total. The Hilbert space of n qubits has dimension 2^n, so operators on it are $2^n \times 2^n$ matrices, which form a vector space of dimension $(2^n)^2 = 4^n$. Since we have exactly 4^n tensor-product operators and they are linearly independent, they **span the entire operator space** — they are a basis.

The practical consequence is the same as in the single-qubit case (Theorem 2.1): any n-qubit error, no matter how complicated, can be written as a *linear combination* of these tensor-product operators — each multiplied by a complex coefficient, exactly as $E = c_0 I + c_1 X + c_2 Y + c_3 Z$ for one qubit. The next subsection makes this parallel explicit.

2.5.2 Why a linear combination, not just one tensor product?

Recall the single-qubit error decomposition:

$$E = c_0\, I \;+\; c_1\, X \;+\; c_2\, Y \;+\; c_3\, Z, \qquad c_i \in \mathbb{C}.$$

Every basis element (I, X, Y, Z) is multiplied by its own complex coefficient. The multi-qubit case is *structurally identical* — the only difference is that the basis elements are now tensor products instead of single Paulis, and there are 4^n of them instead of 4. The general n-qubit error is:

$$E \;=\; \sum_{\mathbf{k}} c_{\mathbf{k}} \qquad \underbrace{P_{k_1}^{(1)} \otimes P_{k_2}^{(2)} \otimes \cdots \otimes P_{k_n}^{(n)}}_{\text{one specific tensor-product error operator}} \qquad , \qquad c_{\mathbf{k}} \in \mathbb{C}.$$

Here the multi-index $\mathbf{k} = (k_1, k_2, \ldots, k_n)$ selects one Pauli for each qubit ($k_j \in \{I, X, Y, Z\}$), and $c_{\mathbf{k}}$ is the complex coefficient for that particular combination — exactly analogous to c_0, c_1, c_2, c_3 in the single-qubit case.

A **single** tensor product (one term with coefficient 1) like $X \otimes Z \otimes I$ describes a specific, "clean" error: qubit 1 flipped, qubit 2 phase-flipped, qubit 3 untouched. A *general* noise process is an arbitrary $2^n \times 2^n$ matrix, which requires a full weighted sum of all 4^n such terms. The coefficients $c_{\mathbf{k}}$ capture how much of each specific error pattern is present.

Syndrome measurement then collapses this weighted sum onto a single term $P_{\mathbf{k}}$, turning the continuous error (with arbitrary $c_{\mathbf{k}}$) into one discrete Pauli tensor product that the code can correct.

2.5.3 Why phases $\{+1, -1, +i, -i\}$ appear

Before asking what the phases are, we should ask a more basic question: *why do we need the Pauli operators to form a group at all?* The answer comes entirely from how syndrome decoding works.

Why a multiplicative group is necessary. Syndrome decoding works by checking, for each stabilizer generator g_i, whether the error E commutes or anticommutes with it:

$$g_i E = +E g_i \;\Rightarrow\; \text{syndrome bit } +1, \qquad g_i E = -E g_i \;\Rightarrow\; \text{syndrome bit } -1.$$

This check involves *multiplying* Pauli operators together. For the syndrome machinery to be well-defined, the result of every such multiplication must stay inside the set we are working with. This is exactly the **closure** property of a group. Without it, $g_i E$ might fall outside the set, and the commutation check would be undefined.

Beyond closure, the other group axioms are equally unavoidable:

- **Identity** ($I \otimes \cdots \otimes I$): the "no error" case must be in the set, otherwise the code cannot handle a perfectly transmitted qubit.
- **Inverses**: every Pauli satisfies $P^2 = I$, so each element is its own inverse — this comes for free from the algebra, but it is essential for composing corrections.
- **Associativity**: matrix multiplication is always associative, so this costs nothing.

The group structure is therefore not imposed by choice — it is the *minimum algebraic requirement* for syndrome decoding to function. The only question is: does the set of tensor-product Paulis already form a group, or do we need to enlarge it?

The closure problem and its fix. Consider the set of all n-qubit tensor products of Paulis *without* any phases:

$$\widetilde{\mathcal{P}}_n = \{P_1 \otimes P_2 \otimes \cdots \otimes P_n \mid P_j \in \{I, X, Y, Z\}\}.$$

Take two elements $A = X \otimes I$ and $B = Y \otimes I$ in $\widetilde{\mathcal{P}}_2$ and multiply them:

$$AB = (X \otimes I)(Y \otimes I) = XY \otimes I = iZ \otimes I.$$

But $iZ \otimes I \notin \widetilde{\mathcal{P}}_2$ because iZ is not one of $\{I, X, Y, Z\}$. **Closure fails.** The set is not a group, so syndrome decoding would be undefined over it.

The source is the single-qubit relation $XY = iZ$: products of distinct Paulis produce phases $\{+1, -1, +i, -i\}$. The minimal fix is to include all four phases explicitly, giving a set that *is* closed under multiplication:

Definition 2.1 (n-qubit Pauli group)**.**

$$\mathcal{P}_n = \{e^{i\phi} P_1 \otimes \cdots \otimes P_n \mid P_j \in \{I, X, Y, Z\},\ \phi \in \{0, \tfrac{\pi}{2}, \pi, \tfrac{3\pi}{2}\}\}.$$

Its size is $|\mathcal{P}_n| = 4^{n+1}$.

Key Point

The phases $\{\pm 1, \pm i\}$ in the Pauli group are not physically observable on their own — a global phase on a quantum state is undetectable. Their importance is *algebraic*: they are the bookkeeping device that keeps the set closed under multiplication, making it a proper group. This group structure is what powers the stabilizer formalism and makes syndrome decoding work cleanly.

Caution

[Phases do not replace coefficients] It is tempting to think that because the Pauli group now includes phases, we no longer need the linear combination $E = \sum_{\mathbf{k}} c_{\mathbf{k}} P_{\mathbf{k}}$ to represent errors. This is not correct. The two things solve entirely different problems:

- The **linear combination with coefficients** $c_{\mathbf{k}}$ answers: *how do we represent an arbitrary error operator?* A general noise process is a continuous object — an arbitrary $2^n \times 2^n$ complex matrix. It requires 4^n continuous complex parameters to describe, which is exactly what the coefficients $c_{\mathbf{k}}$ provide.
- The **phases** $\{\pm 1, \pm i\}$ answer a completely separate question: *how do we keep tensor-product Paulis closed under multiplication?* When two Pauli group elements are multiplied during a syndrome calculation, the result must stay inside the group. Without phases it does not; with them it does.

Even after attaching phases, $\mathcal{P}_n$ is still a *discrete* set of 4^{n+1} elements. A discrete set can never represent a continuous error operator on its own — you still need the continuous coefficients $c_{\mathbf{k}}$. The phases and the coefficients live at completely different levels: the coefficients are for *decomposing errors*, the phases are for *multiplying group elements*.

To see concretely why the phases arise, recall the single-qubit products:

$$XY = iZ, \quad YX = -iZ, \quad YZ = iX, \quad ZY = -iX, \quad ZX = iY, \quad XZ = -iY.$$

Every time two *different* Paulis multiply, a phase $\pm i$ appears. Composing two multi-qubit Pauli operators multiplies them qubit by qubit, accumulating phases from each position:

$$(P_1 \otimes \cdots \otimes P_n)(Q_1 \otimes \cdots \otimes Q_n) = (P_1 Q_1) \otimes \cdots \otimes (P_n Q_n).$$

Each $P_j Q_j$ product can contribute a ± 1 or $\pm i$ phase. The overall phase of the product is the product of all these individual phases.

Worked Example 2.4: Phase accumulation in a product

Compute $(X \otimes Y \otimes Z)(Y \otimes Z \otimes X)$:

$$\begin{aligned} &\text{Qubit 1: } XY = iZ, \quad \text{phase } +i, \\ &\text{Qubit 2: } YZ = iX, \quad \text{phase } +i, \\ &\text{Qubit 3: } ZX = iY, \quad \text{phase } +i. \end{aligned}$$

Overall phase: $(+i)(+i)(+i) = i^3 = -i$.
Result: $-i\,(Z \otimes X \otimes Y)$.
This is why phases must be tracked when composing stabilizer generators in a syndrome calculation.

2.5.4 Do phases matter for error correction?

For *detecting* errors, phases matter only in that they determine commutativity and anticommutativity. A stabilizer g and an error E either commute ($gE = Eg$, giving syndrome $+1$) or anticommute ($gE = -Eg$, giving syndrome -1). The factor of -1 that distinguishes these two cases *is itself a phase*; without consistent phase tracking, the syndrome calculation would be undefined.

For *correcting* errors, global phases on the error operator do not matter: applying P or $-P$ or iP causes the same physical damage to the qubit. The syndrome identifies the Pauli type (which of I, X, Y, Z on each qubit), and the correction applies the same Pauli regardless of phase.

2.5.5 Commutation in the Multi-Qubit Pauli Group

Two n-qubit Paulis A and B either **commute** ($AB = BA$) or **anticommute** ($AB = -BA$). They anticommute if and only if they anticommute on an *odd* number of qubit positions.

Worked Example 2.5: Commutation in $\mathcal{P}_2$

Do $A = X_1 Z_2$ and $B = Z_1 X_2$ commute or anticommute?
At qubit 1: $XZ = -ZX$ — they *anticommute*; sign contribution -1.
At qubit 2: $ZX = -XZ$ — they *anticommute*; sign contribution -1.
Total sign: $(-1)(-1) = +1$. Therefore $AB = BA$: they **commute**.

Do $C = X_1 I_2$ and $D = Z_1 Z_2$ commute?
At qubit 1: $XZ = -ZX$ — anticommute; sign -1.
At qubit 2: $IZ = ZI$ — commute; sign $+1$.
Total sign: $(-1)(+1) = -1$. Therefore $CD = -DC$: they **anticommute**.

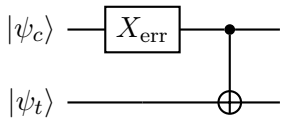
This rule is the engine of syndrome decoding: stabilizer generators are chosen so that each correctable error anticommutes with a *distinct subset* of them, making the pattern of ± 1 measurement outcomes a unique fingerprint of the error type and location.

2.6 Error Propagation in Circuits

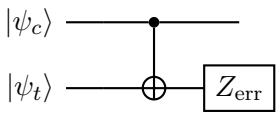
Errors in gates can *propagate* to other qubits:

- An X error on the *control* qubit of a CNOT propagates to both the control and target ($X_c \to X_c X_t$).

- A Z error on the *target* qubit of a CNOT propagates to both ($Z_t \to Z_c Z_t$).



X error on control $\to$ X on both



Z error on target $\to$ Z on both

Figure 2: Error propagation through CNOT: an X error on the control spreads to the target; a Z error on the target spreads to the control.

This motivates careful **fault-tolerant circuit design** covered in Section 8.

2.7 Exercises

Exercise 2.1. Verify equation (2): compute YZ and ZX using matrix multiplication and confirm they equal iX and iY respectively.

Exercise 2.2. Show that the Pauli matrices $\{I, X, Y, Z\}$ form a basis for the space of 2×2 complex matrices. (*Hint*: show they are linearly independent and use a dimension argument.)

Exercise 2.3. For the depolarising channel $\mathcal{E}_{\text{dep}}$ with $p = 0.06$, compute $\mathcal{E}(|0\rangle\langle 0|)$. What is the probability of a bit-flip (X) error? Of a phase-flip (Z) error?

Exercise 2.4. Determine whether the following pairs of 2-qubit Pauli operators commute or anticommute: (a) X_1X_2 and Z_1Z_2; (b) X_1Z_2 and X_1X_2; (c) Y_1Y_2 and X_1Z_2.

Exercise 2.5. For the amplitude damping channel with decay probability γ, verify that $\mathcal{E}(\rho)$ is a valid density matrix when $\rho = \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix}$. What happens to ρ_{11} and ρ_{01} as $\gamma \to 1$? Give a physical interpretation.

Exercise 2.6. Compute the product $(Y_1X_2)(Z_1Y_2)$ in the 2-qubit Pauli group. Identify the overall phase and the resulting Pauli tensor product.

Section Summary

Section 2 key points:

- The three Pauli operators $\{X, Y, Z\}$ describe all single-qubit errors; $\{I, X, Y, Z\}$ form a basis for 2×2 matrices.
- Any physical error decomposes in the Pauli basis; syndrome measurement collapses it to a single Pauli.
- **Kraus operators** $\{K_k\}$ arise from tracing out the environment after system–environment unitary evolution. Each K_k describes one possible "environmental jump"; the channel averages over all jumps. The completeness condition $\sum_k K_k^\dagger K_k = I$ follows from unitarity and ensures probability is conserved.
- **Multi-qubit errors** are arbitrary matrices on n qubits, hence *sums* (not just single products) of Pauli tensor products.
- **Phases** $\{\pm 1, \pm i\}$ appear in the n-qubit Pauli group because products of distinct Pauli matrices produce phases. Without them, the set would not be closed under multiplication. The phases are bookkeeping for algebra, not observable quantities, but they are essential for consistent syndrome calculations.

- Two n-qubit Paulis commute iff they anticommute on an even number of positions.
- Errors propagate through CNOT gates — a key concern for fault-tolerant design.

3 Classical and Quantum Repetition Codes

Intuition

The simplest way to protect a bit is to say it three times. If you tell three people "the answer is **0**" and one of them mishears it as "the answer is **1**", the other two can overrule the mistake. This is a repetition code, and it is the starting point for understanding both classical and quantum error correction.

3.1 Classical 3-Bit Repetition Code

The classical 3-bit repetition code is:

$$0 \longrightarrow 000, \qquad 1 \longrightarrow 111.$$

Decoding by majority vote. If one bit flips, the other two still agree, so majority voting recovers the original bit. A logical error occurs only if two or more bits flip.

Assuming each bit flips independently with probability p:

$$P(\text{logical error}) = \binom{3}{2} p^2(1-p) + \binom{3}{3} p^3$$

$$= 3p^2(1-p) + p^3 = 3p^2 - 2p^3.$$

For small p, this is approximately:

$$P(\text{logical error}) \approx 3p^2.$$

Worked Example 3.1: Classical code improvement

Suppose the physical bit-flip probability is

$$p = 0.01\% = 0.0001.$$

Without encoding, the bit error rate is simply:

$$P(\text{error}) = 0.01\%.$$

With the 3-bit repetition code:

$$P(\text{logical error}) = 3(0.0001)^2 - 2(0.0001)^3$$

$$= 3 \times 10^{-8} - 2 \times 10^{-12} \approx 2.9998 \times 10^{-8}.$$

Expressed as a percentage:

$$P(\text{logical error}) \approx 0.000003\%.$$

So the 3-bit repetition code reduces the error rate by approximately 3333×.

Syndrome table for the classical code. Rather than majority vote, we can identify the error from *parity checks*:

$$s_1 = \text{bit}_1 \oplus \text{bit}_2, \qquad s_2 = \text{bit}_2 \oplus \text{bit}_3.$$

The syndrome (s_1, s_2) tells us exactly which bit (if any) flipped:

Received	**Error**	$s_1 = b_1 \oplus b_2$	$s_2 = b_2 \oplus b_3$
000 (or 111)	None	0	0
100 (or 011)	Flip bit 1	1	0
010 (or 101)	Flip bit 2	1	1
001 (or 110)	Flip bit 3	0	1

3.2 Quantum 3-Qubit Bit-Flip Code

We now lift this idea to the quantum world. The key difference is that we must protect a *superposition*:

$$|0\rangle_L = |000\rangle\,, \tag{4}$$
$$|1\rangle_L = |111\rangle\,, \tag{5}$$
$$|\psi_L\rangle = \alpha\,|000\rangle + \beta\,|111\rangle\,. \tag{6}$$

This is *not* three copies of $|\psi\rangle$ (which the no-cloning theorem forbids). It is an entangled superposition in which α and β live in the global correlations, not in any single qubit.

3.2.1 Encoding Circuit

Starting from $|\psi\rangle\,|0\rangle\,|0\rangle = (\alpha\,|0\rangle + \beta\,|1\rangle)\,|0\rangle\,|0\rangle$, two CNOT gates produce the encoded state:

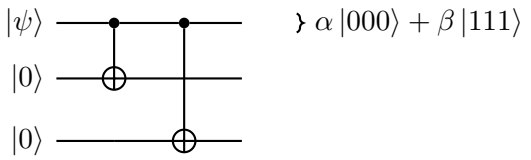


Figure 3: Encoding circuit for the 3-qubit bit-flip code.

Trace through:

$$(\alpha\,|0\rangle + \beta\,|1\rangle)\,|00\rangle \xrightarrow{\text{CNOT}_{1,2}} \alpha\,|000\rangle + \beta\,|110\rangle$$
$$\xrightarrow{\text{CNOT}_{1,3}} \alpha\,|000\rangle + \beta\,|111\rangle\,.$$

3.2.2 Quantum Syndrome Measurement

We measure the **stabilizers** $g_1 = Z_1Z_2$ and $g_2 = Z_2Z_3$ using ancilla qubits. These operators measure the *parity* of pairs of data qubits without touching α or β.

Error	**State after error**	$g_1 = Z_1Z_2$	$g_2 = Z_2Z_3$	**Correction**
None	$\alpha\,\|000\rangle + \beta\,\|111\rangle$	$+1$	$+1$	None
X_1	$\alpha\,\|100\rangle + \beta\,\|011\rangle$	-1	$+1$	Apply X_1
X_2	$\alpha\,\|010\rangle + \beta\,\|101\rangle$	-1	-1	Apply X_2
X_3	$\alpha\,\|001\rangle + \beta\,\|110\rangle$	$+1$	-1	Apply X_3

Figure 4: Syndrome table for the 3-qubit bit-flip code. The syndrome (g_1, g_2) uniquely identifies single-qubit bit-flip errors. Crucially, α and β never appear in the syndrome.

Worked Example 3.2: Full syndrome computation for X_2

Suppose qubit 2 suffers a bit-flip. After encoding, the state is $\alpha\,|000\rangle + \beta\,|111\rangle$. The error X_2 (acting on qubit 2) gives:

$$\text{State after error: } X_2\big(\alpha\,|000\rangle + \beta\,|111\rangle\big) = \alpha\,|010\rangle + \beta\,|101\rangle\,.$$

Measuring $g_1 = Z_1Z_2$:

$$Z_1Z_2\,|010\rangle = Z\,|0\rangle \otimes Z\,|1\rangle \otimes I\,|0\rangle = (+1)(-1)\,|010\rangle = -\,|010\rangle\,,$$
$$Z_1Z_2\,|101\rangle = Z\,|1\rangle \otimes Z\,|0\rangle \otimes I\,|1\rangle = (-1)(+1)\,|101\rangle = -\,|101\rangle\,.$$

Both terms give eigenvalue -1, so the measurement outcome is $g_1 = -1$.
Measuring $g_2 = Z_2Z_3$:

$$Z_2Z_3\,|010\rangle = Z\,|1\rangle \otimes Z\,|0\rangle = (-1)(+1) = -1,$$
$$Z_2Z_3\,|101\rangle = Z\,|0\rangle \otimes Z\,|1\rangle = (+1)(-1) = -1.$$

The measurement outcome is $g_2 = -1$.
Syndrome: $(g_1, g_2) = (-1, -1)$**.** From the table, this corresponds to an X_2 error. We apply X_2 to correct:

$$X_2\big(\alpha\,|010\rangle + \beta\,|101\rangle\big) = \alpha\,|000\rangle + \beta\,|111\rangle\,.$$

The encoded state is perfectly restored. Note that α and β were never measured and are unaffected.

3.2.3 Full Syndrome Circuit

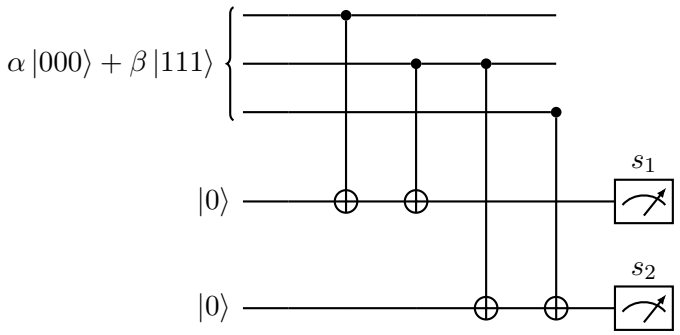


Figure 5: Syndrome measurement circuit for the 3-qubit bit-flip code. Ancilla qubits measure Z_1Z_2 (giving s_1) and Z_2Z_3 (giving s_2) without disturbing $\alpha|000\rangle + \beta|111\rangle$.

3.3 Limitations: Phase Errors Are Not Corrected

The 3-qubit bit-flip code protects against X errors but is *blind* to Z errors.

Worked Example 3.3: Phase flip on qubit 1 is undetected

Suppose a phase-flip Z_1 occurs on the encoded state:

$$Z_1\big(\alpha|000\rangle + \beta|111\rangle\big) = \alpha|000\rangle - \beta|111\rangle.$$

Check the syndrome:

$$\begin{aligned} Z_1Z_2(\alpha|000\rangle - \beta|111\rangle) &= \alpha(+1)(+1)|000\rangle - \beta(-1)(-1)|111\rangle \\ &= \alpha|000\rangle - \beta|111\rangle, \quad g_1 = +1, \\ Z_2Z_3(\alpha|000\rangle - \beta|111\rangle) &= +\alpha|000\rangle - \beta|111\rangle, \quad g_2 = +1. \end{aligned}$$

Syndrome $(+1,+1)$ says "no error" — but the logical state has been changed from $\alpha|0\rangle_L + \beta|1\rangle_L$ to $\alpha|0\rangle_L - \beta|1\rangle_L$. The error is *undetected*.

3.4 Quantum Phase-Flip Code

To correct phase errors, we move to the *Hadamard basis* $\{|+\rangle, |-\rangle\}$:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

In this basis, Z errors become X errors: $Z|+\rangle = |-\rangle$ and $Z|-\rangle = |+\rangle$.

Encode the logical qubit as:

$$|0\rangle_L = |+++\rangle, \qquad |1\rangle_L = |---\rangle,$$

so $|\psi\rangle_L = \alpha|+++\rangle + \beta|---\rangle$.

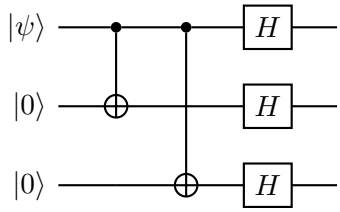


Figure 6: Phase-flip code encoding: first encode with CNOTs (as in the bit-flip code), then apply Hadamard to each qubit.

The stabilizers are now X_1X_2 and X_2X_3 (parity in the Hadamard basis), detecting Z errors. The decoding applies $H^{\otimes 3}$ to convert back, then corrects as before.

Caution

The phase-flip code corrects Z errors only; the bit-flip code corrects X errors only. Neither code corrects both types simultaneously. For complete single-qubit error correction, we need a code that handles both — this is the **Shor 9-qubit code** (Section 4).

3.5 Why This Works Despite No-Cloning

A common confusion: if we cannot clone $|\psi\rangle$, how can we store it in three qubits? The key is that the encoded state $\alpha|000\rangle + \beta|111\rangle$ is *not* three copies of $|\psi\rangle$.

- Tracing out any one qubit of $\alpha|000\rangle + \beta|111\rangle$ gives a *mixed* state (not $|\psi\rangle$).
- The information about α and β is in the *correlations* between the three qubits, not in any single qubit.
- Syndrome measurement reveals only the *parity* of pairs, not α or β individually.

Exercise 3.1. Verify that the syndrome $(g_1, g_2) = (+1, -1)$ correctly identifies an X_3 error by carrying out the eigenvalue calculation explicitly.

Exercise 3.2. Show that the 3-qubit bit-flip code has *two* possible corrective operations for each non-trivial syndrome — for instance, when $(g_1, g_2) = (-1, +1)$ you might apply X_1. Why does it not matter whether we apply X_1 or instead apply X_2X_3 (which has the same syndrome)? (*Hint:* what is the product $X_1 \cdot X_2X_3$ on the encoded subspace?)

Exercise 3.3. Derive the encoding circuit for the quantum phase-flip code by: (a) showing that $H|0\rangle = |+\rangle$ and $H|1\rangle = |-\rangle$; (b) tracing $|\psi\rangle|0\rangle|0\rangle$ through the CNOT+Hadamard circuit to confirm $|0\rangle_L = |+++\rangle$ and $|1\rangle_L = |---\rangle$.

Exercise 3.4. Why does the 3-qubit bit-flip code fail to correct a Y error on qubit 2? Find the syndrome and show that applying the "correction" actually introduces an additional error.

Section Summary

Section 3 key points:

- The quantum 3-qubit bit-flip code encodes $\alpha|0\rangle + \beta|1\rangle$ as $\alpha|000\rangle + \beta|111\rangle$. This is *not* cloning; α, β live in correlations.
- Syndrome measurement identifies which qubit was flipped without disturbing the logical information.

- This code protects against X errors *only*; it is blind to Z (phase) errors.
- The phase-flip code encodes in the Hadamard basis, protecting against Z errors only.
- A code that corrects both X and Z errors requires a new idea: the Shor code.

4 The Shor 9-Qubit Code

Intuition

The **3-qubit bit-flip code** can fix X (bit-flip) errors but is helpless against Z (phase-flip) errors. Conversely, the **3-qubit phase-flip code** fixes Z errors but not X errors. Peter Shor's key insight was to *combine both codes by nesting one inside the other* — a technique called **concatenation**:

1. **Outer layer (phase-flip code):** Encode the single logical qubit into *three* qubits using the phase-flip code. This protects against Z errors across the three groups.
2. **Inner layer (bit-flip code):** Take each of those three qubits and encode *it* into three more qubits using the bit-flip code. This protects against X errors within each group.

The result uses $3\times3 = 9$ physical qubits to store 1 logical qubit, and corrects *any* single-qubit error (X, Z, or Y) on any one of the 9 qubits.

4.1 Step-by-Step Encoding

It is easiest to understand the encoding in two stages.

Stage 1 — Phase-flip code (outer layer)

Start with an arbitrary logical qubit $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$. The 3-qubit phase-flip code maps:

$$\alpha\,|0\rangle + \beta\,|1\rangle \xrightarrow{\text{phase-flip encode}} \alpha\,|+\rangle\,|+\rangle\,|+\rangle + \beta\,|-\rangle\,|-\rangle\,|-\rangle\,,$$

where $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.

Expanding $|+\rangle$ and $|-\rangle$ explicitly:

$$\begin{aligned}\alpha\,|+\rangle\,|+\rangle\,|+\rangle + \beta\,|-\rangle\,|-\rangle\,|-\rangle = {}& \frac{\alpha}{(\sqrt{2})^3}\big(|0\rangle + |1\rangle\big)\big(|0\rangle + |1\rangle\big)\big(|0\rangle + |1\rangle\big) \\ & + \frac{\beta}{(\sqrt{2})^3}\big(|0\rangle - |1\rangle\big)\big(|0\rangle - |1\rangle\big)\big(|0\rangle - |1\rangle\big).\end{aligned}$$

At this point we have **3 qubits**, one per block.

Stage 2 — Bit-flip code (inner layer)

Now apply the 3-qubit bit-flip code independently to each of the 3 qubits from Stage 1:

$$|0\rangle \longrightarrow |000\rangle\,, \qquad |1\rangle \longrightarrow |111\rangle\,.$$

Replacing every $|0\rangle$ with $|000\rangle$ and every $|1\rangle$ with $|111\rangle$ inside the Stage-1 expression gives the final 9-qubit codewords. The logical codewords (subscript L stands for **logical**, distinguishing these 9-qubit encoded states from single physical qubits; $|0\rangle_L$ and $|1\rangle_L$ are *not* individual qubits but entire 9-qubit states that an algorithm treats as a single qubit) are:

$$|0\rangle_L = \frac{1}{2\sqrt{2}}\big(|000\rangle + |111\rangle\big)\big(|000\rangle + |111\rangle\big)\big(|000\rangle + |111\rangle\big), \tag{7}$$

$$|1\rangle_L = \frac{1}{2\sqrt{2}}\big(|000\rangle - |111\rangle\big)\big(|000\rangle - |111\rangle\big)\big(|000\rangle - |111\rangle\big). \tag{8}$$

Key Point

The normalisation factor $\frac{1}{2\sqrt{2}}$ comes from $\left(\frac{1}{\sqrt{2}}\right)^3 = \frac{1}{2\sqrt{2}}$. Each block $\left(|000\rangle \pm |111\rangle\right)$ is not individually normalised; the factor ensures the full 9-qubit state has unit norm.

A general logical state $|\psi\rangle_L = \alpha\,|0\rangle_L + \beta\,|1\rangle_L$ is therefore:

$$|\psi\rangle_L = \frac{\alpha}{2\sqrt{2}}\left(|000\rangle + |111\rangle\right)^{\otimes 3} + \frac{\beta}{2\sqrt{2}}\left(|000\rangle - |111\rangle\right)^{\otimes 3}.$$

4.1.1 Block structure

The 9 qubits are divided into **three blocks of three**:

$$\underbrace{q_1\,q_2\,q_3}_{\text{Block 1}},\quad \underbrace{q_4\,q_5\,q_6}_{\text{Block 2}},\quad \underbrace{q_7\,q_8\,q_9}_{\text{Block 3}}.$$

Within each block a bit-flip code is active; across the three blocks a phase-flip code is active. Figure 7 shows this nested layout.

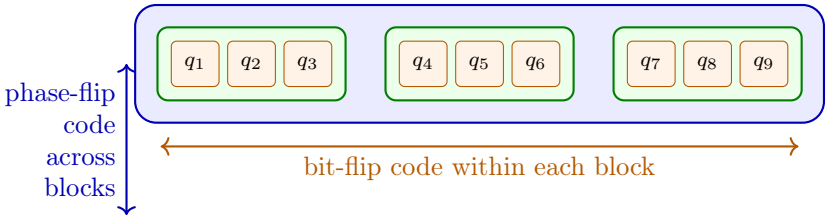


Figure 7: Nested structure of the Shor 9-qubit code. The three inner green boxes each form a 3-qubit *bit-flip* code protecting against X errors. The outer blue box represents the 3-qubit *phase-flip* code protecting against Z errors across the three blocks.

4.1.2 Encoding Circuit and What It Produces

Figure 8 shows the encoding circuit. We trace through it step by step for input $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$.

Initial state.

$$|\Psi_0\rangle = \left(\alpha\,|0\rangle + \beta\,|1\rangle\right) \otimes |0\rangle^{\otimes 8}.$$

Step 1 — Two CNOTs copy q_1 to q_4 and q_7. Qubits q_4 and q_7 become the "leaders" of blocks 2 and 3:

$$|\Psi_1\rangle = \alpha\,|0\rangle\,|00\rangle\,|0\rangle\,|00\rangle\,|0\rangle\,|00\rangle + \beta\,|1\rangle\,|00\rangle\,|1\rangle\,|00\rangle\,|1\rangle\,|00\rangle.$$

(Within each block, only the leader qubit is set; the two ancillas are still $|0\rangle$.)

Step 2 — Hadamard on each block leader q_1, q_4, q_7. Each leader transforms as $|0\rangle \to |+\rangle$ and $|1\rangle \to |-\rangle$:

$$\begin{aligned}|\Psi_2\rangle &= \alpha\,|+\rangle\,|00\rangle\,|+\rangle\,|00\rangle\,|+\rangle\,|00\rangle + \beta\,|-\rangle\,|00\rangle\,|-\rangle\,|00\rangle\,|-\rangle\,|00\rangle \\ &= \frac{\alpha}{2\sqrt{2}}\left(|0\rangle + |1\rangle\right)|00\rangle\ \left(|0\rangle + |1\rangle\right)|00\rangle\ \left(|0\rangle + |1\rangle\right)|00\rangle \\ &\quad + \frac{\beta}{2\sqrt{2}}\left(|0\rangle - |1\rangle\right)|00\rangle\ \left(|0\rangle - |1\rangle\right)|00\rangle\ \left(|0\rangle - |1\rangle\right)|00\rangle.\end{aligned}$$

Step 3 — Two CNOTs within each block spread the leader to its two ancillas. Inside each block: $|0\rangle|00\rangle \to |000\rangle$ and $|1\rangle|00\rangle \to |111\rangle$. The full 9-qubit output state is:

$$|\Psi_{\text{enc}}\rangle = \frac{\alpha}{2\sqrt{2}}\big(|000\rangle + |111\rangle\big)\big(|000\rangle + |111\rangle\big)\big(|000\rangle + |111\rangle\big) + \frac{\beta}{2\sqrt{2}}\big(|000\rangle - |111\rangle\big)\big(|000\rangle - |111\rangle\big)\big(|000\rangle - |111\rangle\big) \tag{9}$$

which is exactly $\alpha|0\rangle_L + \beta|1\rangle_L$ as required.

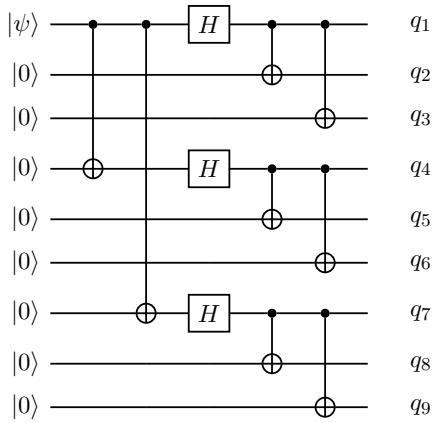


Figure 8: Shor code encoding circuit. **Step 1** (columns 1–2): two CNOTs copy the logical qubit to q_4 and q_7, forming the phase-flip backbone across the three blocks. **Step 2** (column 3): Hadamards on q_1, q_4, q_7 create $(\pm)$-basis superpositions in each block leader. **Step 3** (columns 4–5): CNOTs within each block spread the leader to its two ancillas, producing the $|000\rangle \pm |111\rangle$ structure. Output state: Eq. (9).

4.2 Stabilizer Generators

The Shor code is a **stabilizer code** (see Section 6). A stabilizer is an operator g such that $g|\psi_L\rangle = +|\psi_L\rangle$ for every valid codeword. Errors are detected because an error E on a single qubit will *anticommute* with some stabilizer generators, flipping their eigenvalue from $+1$ to -1.

The 8 independent generators divide into two families.

4.2.1 Bit-flip stabilizers (within each 3-qubit block)

$$g_1 = Z_1Z_2, \quad g_2 = Z_2Z_3, \quad g_3 = Z_4Z_5, \quad g_4 = Z_5Z_6, \quad g_5 = Z_7Z_8, \quad g_6 = Z_8Z_9.$$

Intuition

Within block 1 the valid states are $|000\rangle$ and $|111\rangle$. Both satisfy $Z_1Z_2|000\rangle = (+1)(+1)|000\rangle = +|000\rangle$ and $Z_1Z_2|111\rangle = (-1)(-1)|111\rangle = +|111\rangle$. If an X error flips qubit 2, the state becomes $|010\rangle$ or $|101\rangle$, and Z_1Z_2 gives eigenvalue -1, signalling the error. Each Z_iZ_j acts as a *parity check* in the computational basis within its block.

4.2.2 Phase-flip stabilizers (across blocks)

$$g_7 = X_1X_2X_3X_4X_5X_6, \qquad g_8 = X_1X_2X_3X_7X_8X_9.$$

Intuition

These 6-qubit operators compare the *relative phase* between blocks. In $|0\rangle_L$ all three blocks have a $+$ sign, so both g_7 and g_8 give $+1$. A Z error on any qubit in block 1 flips the relative

sign between block 1 and the others, causing g_7 and g_8 to return -1.

Key Point

Eight generators for 9 physical qubits $\Rightarrow 2^{9-8} = 2$ code states, encoding exactly **1 logical qubit**. All 8 generators commute pairwise — a necessary condition for a valid stabilizer code.

4.3 Error Correction: How It Works

When a physical error occurs, we measure all 8 stabilizers without disturbing the logical information. The pattern of ± 1 outcomes — the **syndrome** — uniquely identifies the error, which we then undo with a correction gate.

Worked Example 4.1: Correcting a bit-flip (X_5) on qubit 5

Suppose qubit 5 suffers an X error.

Why do generators in block 2 fire? Qubit 5 belongs to block 2. The bit-flip stabilizers for block 2 are $g_3 = Z_4Z_5$ and $g_4 = Z_5Z_6$. Since Z and X anticommute ($ZX = -XZ$), measuring g_3 or g_4 after the error on qubit 5 flips their sign.

Syndrome computation:

Generator	Relation to X_5	Outcome
$g_1 = Z_1Z_2$	acts on qubits 1,2 only — commutes	$+1$
$g_2 = Z_2Z_3$	acts on qubits 2,3 only — commutes	$+1$
$g_3 = Z_4Z_5$	Z_5 anticommutes with X_5	$\mathbf{-1}$
$g_4 = Z_5Z_6$	Z_5 anticommutes with X_5	$\mathbf{-1}$
$g_5 = Z_7Z_8$	acts on qubits 7,8 only — commutes	$+1$
$g_6 = Z_8Z_9$	acts on qubits 8,9 only — commutes	$+1$
$g_7 = X_1X_2X_3X_4X_5X_6$	X_5 commutes with X_5	$+1$
$g_8 = X_1X_2X_3X_7X_8X_9$	does not act on qubit 5	$+1$

Syndrome: $(g_1, \ldots, g_8) = (+, +, -, -, +, +, +, +)$.

Interpretation: Only g_3 and g_4 flipped — both share qubit 5. Within block 2 both parity checks failed, pointing to the middle qubit.

Correction: Apply X_5.

Worked Example 4.2: Correcting a phase-flip (Z_2) on qubit 2

Suppose qubit 2 suffers a Z error.

Why do phase stabilizers fire? Z_2 anticommutes with X_2. Qubit 2 appears as an X_2 factor in both g_7 and g_8.

Syndrome computation:

Generator	Relation to Z_2	Outcome
$g_1 = Z_1Z_2$	Z_2 commutes with Z_2 ($ZZ = ZZ$)	$+1$
$g_2 = Z_2Z_3$	Z_2 commutes with Z_2	$+1$
g_3, g_4, g_5, g_6	act on blocks 2 and 3 only	$+1$
$g_7 = X_1X_2X_3X_4X_5X_6$	X_2 anticommutes with Z_2	$\mathbf{-1}$
$g_8 = X_1X_2X_3X_7X_8X_9$	X_2 anticommutes with Z_2	$\mathbf{-1}$

Syndrome: $(g_1, \ldots, g_8) = (+, +, +, +, +, +, -, -)$.

Interpretation: Both g_7 and g_8 flipped. g_7 involves blocks 1&2 vs. block 3; g_8 involves blocks 1&2 vs. block 3. Both firing indicates a phase error in **block 1**. The bit-flip stabilizers all gave $+1$, confirming there is no bit-flip — the error is a pure phase flip.

Correction: Apply Z to any one qubit in block 1 (e.g. Z_1, Z_2, or Z_3) to restore the correct phase.

Note: The syndrome identifies *which block* has the phase error but not *which specific qubit* within the block. This is fine: a Z error on any qubit within a block has the same effect on the encoded state, so applying Z to any qubit in that block corrects it.

4.4 Correcting Arbitrary Single-Qubit Errors

Any single-qubit error on any of the 9 qubits can be written as $E = c_0I + c_1X + c_2Y + c_3Z$. Because error correction is linear, it suffices to show we can correct each Pauli separately.

1. I **(no error):** Syndrome is all $+1$; no correction needed.
2. X_j **(bit-flip on qubit j):** The two bit-flip stabilizers within the block containing j both return -1; all others return $+1$. The syndrome uniquely identifies the block and the position of j within it. Apply X_j to correct.
3. Z_j **(phase-flip on qubit j):** Z_j commutes with all Z-type bit-flip stabilizers, so g_1–g_6 return $+1$. Z_j anticommutes with the X_j factor in g_7 and/or g_8, flipping those outcomes. The pattern of g_7, g_8 identifies which block is affected. Apply Z to any qubit in that block.
4. $Y_j = iX_jZ_j$ **(combined error on qubit j):** Y triggers *both* the bit-flip and the phase-flip syndromes simultaneously. Apply X_j to fix the bit-flip and Z_j to fix the phase-flip (or equivalently apply Y_j).

Key Point

The syndromes for X, Z, and Y errors on every qubit are all **distinct**, covering every non-trivial syndrome pattern. The Shor code therefore corrects **any single-qubit error on any of the 9 qubits** while preserving the logical qubit perfectly.

4.5 Logical Operators

The operators that act *logically* on the encoded qubit without being detected by the stabilizers are:

$$\bar{X} = X_1X_2X_3X_4X_5X_6X_7X_8X_9 = X^{\otimes 9},$$
$$\bar{Z} = Z_1Z_2Z_3.$$

Intuition

$\bar{X} = X^{\otimes 9}$ commutes with all 8 stabilizer generators and flips the logical bit: $|0\rangle_L \leftrightarrow |1\rangle_L$. $\bar{Z} = Z_1 Z_2 Z_3$ commutes with all stabilizers and introduces a relative phase between $|0\rangle_L$ and $|1\rangle_L$. Any product of one Z from each block works equally well, e.g. $Z_1 Z_4 Z_7$ or $Z_2 Z_5 Z_8$.

Exercise 4.1. Verify that g_7 and g_8 commute. (*Hint:* they share 3 common X-positions; count how many positions they anticommute.)

Exercise 4.2. Show that $\bar{Z} = Z_1 Z_2 Z_3$ commutes with all 8 stabilizer generators but is not itself in the stabilizer group.

Exercise 4.3. Why does the Shor code use 9 qubits and not 7? Research (or argue from first principles) why the code distance must be at least 3 to correct any single-qubit error.

Section Summary

Section 4 key points:

- The Shor code combines a phase-flip code (3 blocks) with a bit-flip code (within each block) to correct any single-qubit error.
- It uses 9 physical qubits to encode 1 logical qubit.
- 8 stabilizer generators (6 ZZ-type, 2 $XXXXXX$-type) produce 8 syndrome bits, uniquely identifying all single-qubit Pauli errors.
- Any continuous single-qubit error is corrected via discretisation.

5 The Steane $[7, 1, 3]$ Code

Intuition

The Shor code encodes 1 logical qubit using 9 physical qubits. That works, but it feels wasteful. Can we do better? The **Steane code** achieves the same error-correction power with only **7 physical qubits**. To understand how, we first need to look at a classical error-correcting code called the **Hamming code**, which has been used in computer memory chips since the 1950s. The Steane code is essentially the Hamming code "lifted" to the quantum world.

5.1 Step 1 — The Classical Hamming $[7, 4, 3]$ Code

5.1.1 What is a classical code?

Imagine you want to send 4 bits of information over a noisy channel that might flip one of your bits. If you send the 4 bits raw, a single flip gives you wrong information and you cannot even tell where the error occurred.

A **classical error-correcting code** solves this by adding *redundant* bits (called **parity bits**) before sending. The Hamming code adds 3 extra parity bits to your 4 data bits, producing a 7-bit **codeword**. The key promise is:

> *"If at most one of the 7 transmitted bits is flipped, the receiver can figure out* which *bit was flipped and correct it."*

The parameters of the code are written $[n, k, d] = [7, 4, 3]$:

- $n = 7$: total bits in a codeword,
- $k = 4$: data (information) bits,
- $d = 3$: minimum distance — any two valid codewords differ in at least 3 positions, so $\lfloor (3-1)/2 \rfloor = 1$ error can always be corrected.

5.1.2 Two complementary tools: G and H

The Hamming code is described by two matrices that work as a pair.

Matrix	Role	How you use it
Generator matrix G (4×7)	**Encoding**: turns your message into a valid codeword.	Input: 4-bit message $\mathbf{m}$. Compute $\mathbf{c} = \mathbf{m}G \pmod 2$. Output: valid 7-bit codeword $\mathbf{c}$.
Parity-check matrix $H (3 \times 7)$	**Checking/decoding**: detects and locates errors.	Input: received 7-bit word $\mathbf{r}$. Compute syndrome $\mathbf{s} = H\mathbf{r}^T \pmod 2$. If $\mathbf{s} = \mathbf{0}$: no error. Otherwise: $\mathbf{s}$ identifies the flipped bit.

Think of G as the **sender's tool** and H as the **receiver's tool**. They are designed together so that every codeword produced by G is automatically accepted by H:

$$HG^T = 0 \pmod 2.$$

This guarantees that if no error occurs, the syndrome is always zero.

5.1.3 Valid codewords: not every 7-bit string qualifies

Out of $2^7 = 128$ possible 7-bit strings, only $2^4 = 16$ are valid codewords. Those 16 are precisely the strings that pass all three parity checks imposed by H.

The parity-check constraints are *not* automatically satisfied by arbitrary bit strings — they are conditions that the encoder deliberately builds into the codeword using G. This is the foundation of the whole scheme:

- The **sender** uses G to construct one of the 16 valid codewords from the 4-bit message.
- The **receiver** uses H to check whether the received string is still one of those 16. If it is not, at least one bit was flipped in transit.

5.1.4 The generator matrix: constructing a codeword

The generator matrix for this Hamming code is:

$$G = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

The last four columns of G form a 4×4 identity block. This means bits 4–7 of every codeword are exactly your 4 original data bits, and the first three bits are the parity bits that G computes for you.

To encode a message $\mathbf{m} = (m_1, m_2, m_3, m_4)$, XOR together the rows of G wherever the corresponding message bit is 1:

$$\mathbf{c} = \mathbf{m}\,G \pmod 2 = m_1\,(\text{row}_1) \oplus m_2\,(\text{row}_2) \oplus m_3\,(\text{row}_3) \oplus m_4\,(\text{row}_4).$$

Worked Example 5.1: Encoding $\mathbf{m} = (0,0,0,0)$ and $\mathbf{m} = (1,1,1,1)$

Message $\mathbf{m} = (0,0,0,0)$. All message bits are 0, so no rows are selected.

$$\mathbf{c} = (0,0,0,0,0,0,0).$$

The all-zeros message gives the all-zeros codeword.

Message $\mathbf{m} = (1,1,1,1)$. All message bits are 1, so XOR all four rows of G:

$$\begin{aligned} \text{row}_1 &= (1,1,0,1,0,0,0) \\ \text{row}_2 &= (0,1,1,0,1,0,0) \\ \text{row}_3 &= (1,0,1,0,0,1,0) \\ \text{row}_4 &= (1,1,1,0,0,0,1) \\ \hline \mathbf{c} &= (1,1,1,1,1,1,1) \pmod 2. \end{aligned}$$

Notice that bits 4–7 are $(1,1,1,1)$, which are the original data bits, and bits 1–3 are the computed parity bits $(1,1,1)$.

Verify using H:

$$H = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

For $\mathbf{c} = (1,1,1,1,1,1,1)$, every entry is 1, so we sum all 7 columns of H:

$$\begin{aligned} \text{Row 1 check:} \quad & 1+0+0+1+0+1+1 = 4 \equiv 0 \pmod 2. \\ \text{Row 2 check:} \quad & 0+1+0+1+1+0+1 = 4 \equiv 0 \pmod 2. \\ \text{Row 3 check:} \quad & 0+0+1+0+1+1+1 = 4 \equiv 0 \pmod 2. \end{aligned}$$

All three checks pass, confirming $(1,1,1,1,1,1,1)$ is a valid codeword. ✓

5.1.5 The parity-check matrix H in detail

$$H = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

Each **row** specifies one parity check. The 1s in a row tell you which bit positions are included in that check. Row i of $H\mathbf{c}^T$ computes the XOR of the bits at those positions; a valid codeword must give 0 on every row.

- **Row 1** checks bits 1, 4, 6, 7: $c_1 \oplus c_4 \oplus c_6 \oplus c_7 = 0$.
- **Row 2** checks bits 2, 4, 5, 7: $c_2 \oplus c_4 \oplus c_5 \oplus c_7 = 0$.
- **Row 3** checks bits 3, 5, 6, 7: $c_3 \oplus c_5 \oplus c_6 \oplus c_7 = 0$.

These three equations are the constraints that G builds into every codeword it produces. A random 7-bit string will almost certainly violate at least one of them.

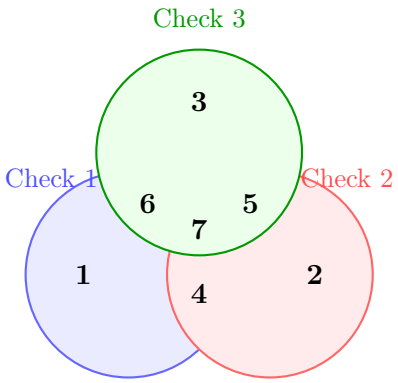


Figure 9: The three parity-check groups of the Hamming code. Each circle shows which bit positions participate in that check. Bit 7 sits in the centre because it is included in all three checks.

5.1.6 The syndrome: pinpointing the error

Suppose a valid codeword $\mathbf{c}$ is sent but a single bit (bit j) is flipped in transit. The receiver gets $\mathbf{r} = \mathbf{c} \oplus \mathbf{e}$, where $\mathbf{e}$ is the all-zero vector except for a 1 in position j. The receiver computes the **syndrome**:

$$\mathbf{s} = H\mathbf{r}^T = H(\mathbf{c} \oplus \mathbf{e})^T = \underbrace{H\mathbf{c}^T}_{=\mathbf{0}} \oplus H\mathbf{e}^T = H\mathbf{e}^T \pmod 2.$$

Because $H\mathbf{c}^T = \mathbf{0}$ for any valid codeword, the syndrome depends *only on the error*, not on which message was sent. This is what makes decoding unambiguous.

Since $\mathbf{e}$ has exactly one 1 (at position j), the product $H\mathbf{e}^T$ picks out exactly the j-th column of H.

The key property: the 7 columns of H are all 7 possible nonzero 3-bit strings, each one different. So every single-bit error produces a *unique* syndrome, and the syndrome tells the receiver exactly which bit was flipped.

Worked Example 5.2: Error detection and correction

Setup. The sender transmits the all-zeros codeword $\mathbf{c} = (0,0,0,0,0,0,0)$ (message $\mathbf{m} = (0,0,0,0)$). During transmission, **bit 4 is flipped**. The receiver gets $\mathbf{r} = (0,0,0,\mathbf{1},0,0,0)$.

Step 1: Compute the syndrome. Only bit 4 of $\mathbf{r}$ is 1, so the syndrome is just column 4 of H:

$$\mathbf{s} = H\mathbf{r}^T = 1 \cdot \underbrace{\begin{pmatrix}1\\1\\0\end{pmatrix}}_{\text{column 4 of } H} = \begin{pmatrix}1\\1\\0\end{pmatrix}.$$

Step 2: Identify the flipped bit. We look up which column of H equals the syndrome $(1,1,0)^T$:

Bit position j	1	2	3	4	5	6	7
Column j of H	$\begin{pmatrix}1\\0\\0\end{pmatrix}$	$\begin{pmatrix}0\\1\\0\end{pmatrix}$	$\begin{pmatrix}0\\0\\1\end{pmatrix}$	$\begin{pmatrix}1\\1\\0\end{pmatrix}$	$\begin{pmatrix}0\\1\\1\end{pmatrix}$	$\begin{pmatrix}1\\0\\1\end{pmatrix}$	$\begin{pmatrix}1\\1\\1\end{pmatrix}$

The syndrome $(1,1,0)^T$ matches **column 4**. The error is on **bit 4**.

Step 3: Correct the error. Flip bit 4 of $\mathbf{r}$ back to 0, recovering $\mathbf{c} = (0,0,0,0,0,0,0)$ and therefore message $\mathbf{m} = (0,0,0,0)$.

Why does this always work? The 7 columns of H are all 7 nonzero 3-bit strings, each distinct. Every single-bit error therefore produces a unique syndrome, and the syndrome unambiguously identifies the flipped bit.

5.2 Step 2 — The CSS Construction: Going Quantum

5.2.1 Why can't we just use the classical code directly?

In quantum computing, qubits can suffer two independent kinds of single-qubit error:

- X **errors** (bit-flip): $|0\rangle \leftrightarrow |1\rangle$. Analogous to classical bit-flips.
- Z **errors** (phase-flip): $|+\rangle \leftrightarrow |-\rangle$. No classical analogue.

We need a code that detects *both* types simultaneously without collapsing the encoded quantum state.

5.2.2 The CSS idea: two codes in one

Calderbank, Shor, and Steane discovered that if a classical code satisfies a certain self-compatibility condition, it can be doubled up to protect against both kinds of quantum errors at once.

Definition 5.1 (CSS code)**.** Given classical linear codes $C_1 \supseteq C_2$ with parity-check matrices H_1 and H_2, the **CSS code** $\mathrm{CSS}(C_1, C_2)$ has stabiliser generators:

X-type: one generator per row of H_2, placing X on each qubit where the row has a 1,

Z-type: one generator per row of H_1, placing Z on each qubit where the row has a 1.

All generators commute if and only if $H_1 H_2^T = 0 \pmod 2$.

Why does $H_1 H_2^T = 0$ guarantee commutativity? An X-generator and a Z-generator commute if and only if they share an *even* number of qubits (two anticommutations on the same qubit cancel each other, restoring a $+1$ sign). The (i, j)-entry of $H_1 H_2^T$ counts (mod 2) how many qubits are shared by the i-th Z-generator and the j-th X-generator. Setting this to 0 forces every such pair to overlap evenly, and hence to commute.

5.3 Step 3 — Building the Steane Code

The Steane code uses $H_1 = H_2 = H$ (the same Hamming parity-check matrix for both X- and Z-type generators).

5.3.1 Checking the CSS condition

We need $HH^T = 0 \pmod 2$. As a spot-check, row 1 dotted with row 2:

$$(1,0,0,1,0,1,1) \cdot (0,1,0,1,1,0,1)^T = 0+0+0+1+0+0+1 = 2 \equiv 0 \pmod 2.$$

All nine entries work out to 0 (mod 2) (a special property of the Hamming code called **self-orthogonality**), so the CSS condition is satisfied.

5.3.2 The six stabiliser generators

Each of the 3 rows of H gives one X-generator and one Z-generator, for 6 generators in total:

Row	Positions of 1s	X-generator	Z-generator
1	1, 4, 6, 7	$g_1 = X_1X_4X_6X_7$	$g_4 = Z_1Z_4Z_6Z_7$
2	2, 4, 5, 7	$g_2 = X_2X_4X_5X_7$	$g_5 = Z_2Z_4Z_5Z_7$
3	3, 5, 6, 7	$g_3 = X_3X_5X_6X_7$	$g_6 = Z_3Z_5Z_6Z_7$

These 6 stabilisers reduce the 7-qubit Hilbert space from $2^7 = 128$ dimensions to $2^{7-6} = 2$ dimensions — just enough room for one logical qubit.

5.3.3 Commutation, anticommutation, and why they matter

Before explaining how errors are detected, we need to understand one crucial point: what it means for two operators to **commute** or **anticommute**, and why that determines the measurement outcome.

The basic Pauli anticommutation relation. The single-qubit Pauli operators X and Z satisfy:

$$XZ = -ZX.$$

Applying X then Z gives the *opposite sign* to applying Z then X. We say X and Z **anticommute**. By contrast, $ZZ = ZZ$ and $XX = XX$, so same-type pairs **commute**.

Stabilisers and measurement outcomes. The encoded state $|\psi\rangle$ is defined to be a $+1$ eigenstate of every stabiliser generator g:

$$g\,|\psi\rangle = +1 \cdot |\psi\rangle.$$

Measuring g on the error-free state always gives $+1$ — the "all clear" signal.

How an error changes the measurement outcome. Suppose error E has occurred, so the state is now $E|\psi\rangle$. Measuring g on this state gives:

Relationship between E and g	Outcome	Why
$gE = Eg$ (commute)	$+1$	$g(E\|\psi\rangle) = E(g\|\psi\rangle) = E(+1\|\psi\rangle) = +1 \cdot (E\|\psi\rangle)$
$gE = -Eg$ (anticommute)	-1	$g(E\|\psi\rangle) = -E(g\|\psi\rangle) = -E(+1\|\psi\rangle) = -1 \cdot (E\|\psi\rangle)$

A -1 outcome is the flag that an error has occurred. Crucially, this measurement does *not* collapse or disturb the encoded logical information — it only reveals which error happened.

The rule for multi-qubit stabilisers. On a single qubit, X and Z anticommute; operators on *different* qubits always commute. Therefore, for a multi-qubit error E and stabiliser g:

Situation	Outcome of measuring g
E and g share no qubit	$+1$ (commute)
E and g share qubits, but same Pauli type (X with X, or Z with Z)	$+1$ (commute)
E and g share an *odd* number of qubits where their Pauli types differ (X vs Z)	-1 (anticommute)

The punchline: a stabiliser flips to -1 precisely when it "sees" the error on an odd number of qubits. By recording which generators give -1, we obtain a binary syndrome — exactly like the Hamming syndrome, but now operating on quantum operators.

5.3.4 How errors are detected

Detecting a Z error (phase-flip). A Z error on qubit j behaves as follows against each generator:

- An X-generator that *includes* qubit j has an X on qubit j. Since $ZX = -XZ$, they anticommute there $\Rightarrow$ outcome -1.
- An X-generator that *excludes* qubit j has no X on qubit j, so it commutes with Z_j $\Rightarrow$ outcome $+1$.
- All Z-generators commute with Z_j (same type everywhere) $\Rightarrow$ outcome $+1$ always.

The pattern of -1 outcomes among g_1, g_2, g_3 is precisely the Hamming syndrome for an error on bit j. The Hamming decoder identifies j.

Detecting an X error (bit-flip). By symmetry (swap X and Z throughout): an X error on qubit j anticommutes with every Z-generator that includes qubit j, and commutes with everything else. The Z-syndrome identifies j.

Worked Example 5.3: Detecting a Z-error on qubit 3

The error. A phase-flip Z_3 occurs on qubit 3.

Measure the X-type generators. Z_3 anticommutes with X only on qubit 3.

$$\begin{aligned} g_1 = X_1X_4X_6X_7: &\quad 3 \notin \{1,4,6,7\} \Rightarrow \text{commutes} \Rightarrow \text{outcome } +1. \\ g_2 = X_2X_4X_5X_7: &\quad 3 \notin \{2,4,5,7\} \Rightarrow \text{commutes} \Rightarrow \text{outcome } +1. \\ g_3 = X_3X_5X_6X_7: &\quad 3 \in \{3,5,6,7\} \Rightarrow \text{anticommutes} \Rightarrow \text{outcome } -1. \end{aligned}$$

Measure the Z-type generators. Z_3 commutes with all Z-generators (same Pauli type), so all give $+1$.

Decode the syndrome. Map $+1 \to 0$ and $-1 \to 1$:

$$(g_1, g_2, g_3) = (+1, +1, -1) \longrightarrow (0, 0, 1).$$

Column 3 of H is $(0,0,1)^T$, which matches exactly. The error is on **qubit 3**.

Correction. Apply Z_3. The encoded state is restored without ever measuring the logical qubit directly.

Key Point

The Steane code runs the Hamming syndrome decoder *twice in parallel* — once to find Z errors (using X-generator outcomes) and once to find X errors (using Z-generator outcomes). The two syndromes are completely independent and do not interfere with each other.

5.4 Code Parameters and Logical Operators

5.4.1 Parameters

Parameter	Value	What it means
n	7	Total physical qubits used.
k	1	Logical qubits encoded. $k = n -$ (no. of generators) $= 7 - 6 = 1$.
d	3	Code distance. We can correct $\lfloor (d-1)/2 \rfloor = 1$ arbitrary single-qubit error.

5.4.2 Logical operators

Every quantum code must have **logical operators** that act on the encoded qubit without being detected by any stabiliser. For the Steane code:

$$\bar{X} = X^{\otimes 7}, \qquad \bar{Z} = Z^{\otimes 7}.$$

Both act on all 7 qubits, consistent with $d = 3$: an adversary must corrupt at least 3 qubits simultaneously to implement an undetectable logical operation.

5.5 Transversal Gates: A Major Advantage

A logical gate is **transversal** if it is implemented by applying the same operation independently to each physical qubit. Transversal gates cannot spread errors between qubits during the gate, making them inherently fault-tolerant.

Logical gate	Physical implementation	Why it works
$\bar{H}$ (Hadamard)	$H^{\otimes 7}$	The Steane code is self-dual ($H_X = H_Z = H$), so H on every qubit swaps X-generators with Z-generators, mapping the code back to itself.
$\bar{S}$ (Phase, $S = \mathrm{diag}(1, i)$)	$S^{\otimes 7}$	S maps $X \to Y = iXZ$ and $Z \to Z$, preserving the CSS stabiliser structure.
$\overline{\mathrm{CNOT}}$	$\mathrm{CNOT}^{\otimes 7}$	Bitwise CNOT between two 7-qubit Steane code blocks.

Together, $\bar{H}$, $\bar{S}$, $\overline{\mathrm{CNOT}}$ generate the **Clifford group** — powerful but not universal.

Caution

The T gate ($T = \mathrm{diag}(1, e^{i\pi/4})$) is *not* transversal in the Steane code: $T^{\otimes 7}$ does not implement a logical $\bar{T}$. A fault-tolerant $\bar{T}$ requires **magic state distillation** (Section 8.4), the single largest overhead cost in fault-tolerant quantum computing.

5.6 Summary: Shor vs. Steane

	Shor code	Steane code
Physical qubits n	9	7
Logical qubits k	1	1
Code distance d	3	3
Errors correctable	1	1
Transversal Clifford gates?	Partial	Yes (full Clifford group)
Key idea	Concatenation	CSS + Hamming code

Both codes correct any single-qubit error, but the Steane code does so with fewer qubits and a richer set of transversal gates, making it the more practical choice and a foundational building block for modern fault-tolerant architectures.

Exercise 5.1. Verify that $HH^T = 0 \pmod 2$ for the Hamming matrix H given above. This is the CSS construction validity condition.

Exercise 5.2. Compute the Z-syndrome (outcomes of g_4, g_5, g_6) when an X-error occurs on qubit 5. Confirm it correctly identifies qubit 5.

Exercise 5.3. Explain why the Steane code needs only 7 qubits while the Shor code needs 9 to correct the same set of errors. What property of the Hamming code makes this possible?

Section Summary

Section 5 key points:

- The Steane code is a $[\![7, 1, 3]\!]$ CSS code derived from the classical Hamming $[7, 4, 3]$ code.

- Its 6 stabilizer generators (3 X-type, 3 Z-type) come directly from the rows of the Hamming parity-check matrix.
- X-type and Z-type generators always commute in a CSS code.
- The code admits transversal Hadamard, Phase, and CNOT — the full Clifford group — but not the T gate.

6 The Stabilizer Formalism

Intuition

The 3-qubit, Shor, and Steane codes all share a common structure: there is a set of multi-qubit Pauli operators (the *stabilizers*) that all codewords satisfy simultaneously. Errors shift codewords out of this "eigenspace", and measuring the stabilizers — without touching the logical information — tells you what went wrong. The stabilizer formalism is the mathematical framework that unifies all these examples and provides a systematic way to construct and analyse quantum codes.

6.1 The Pauli Group (Reminder)

The n-qubit Pauli group $\mathcal{P}_n$ was defined in Section 2. Its key properties:

- Every element squares to $\pm I$.
- Any two elements either commute or anticommute.
- Elements can be described efficiently: an n-qubit Pauli is specified by two n-bit strings (a, b) where $a_j = 1$ means X_j appears and $b_j = 1$ means Z_j appears (with $Y_j = iX_jZ_j$ when both are 1), plus a phase.

6.2 Definition of a Stabilizer Code

Definition 6.1 (Stabilizer group and code space)**.** An abelian subgroup $S \leq \mathcal{P}_n$ not containing $-I$ is called a **stabilizer group**. The associated **code space** is the simultaneous $+1$ eigenspace:

$$\mathcal{C}(S) = \left\{ |\psi\rangle \in (\mathbb{C}^2)^{\otimes n} \;\middle|\; s\,|\psi\rangle = +\,|\psi\rangle\,,\ \forall s \in S \right\}.$$

If S is generated by $n-k$ independent generators $\{g_1, \ldots, g_{n-k}\}$, then $\dim \mathcal{C}(S) = 2^k$, encoding k logical qubits into n physical qubits.

Worked Example 6.1: Verifying the 3-qubit code space

The 3-qubit bit-flip code has stabilizer $S = \langle Z_1Z_2, Z_2Z_3 \rangle$. We check that $|000\rangle$ and $|111\rangle$ are in the code space:

$$\begin{aligned} Z_1Z_2\,|000\rangle &= (+1)(+1)\,|000\rangle = +\,|000\rangle\,, \\ Z_2Z_3\,|000\rangle &= (+1)(+1)\,|000\rangle = +\,|000\rangle\,, \\ Z_1Z_2\,|111\rangle &= (-1)(-1)\,|111\rangle = +\,|111\rangle\,, \\ Z_2Z_3\,|111\rangle &= (-1)(-1)\,|111\rangle = +\,|111\rangle\,. \end{aligned}$$

Any superposition $\alpha\,|000\rangle + \beta\,|111\rangle$ is therefore also in the code space. We have $n = 3$, 2 generators, so $k = 3 - 2 = 1$ logical qubit encoded. Correct!

6.3 Logical Operators

Definition 6.2 (Normaliser and logical operators)**.** The **normaliser** of S in $\mathcal{P}_n$ is:

$$N(S) = \{P \in \mathcal{P}_n \mid PSP^{-1} = S\} = \{P \in \mathcal{P}_n \mid [P, s] = 0,\ \forall s \in S\}.$$

Logical operators are elements of $N(S) \setminus S$. They preserve the code space ($P\mathcal{C} = \mathcal{C}$) but act non-trivially on the encoded qubits.

- Logical $\bar{X}_i$ and $\bar{Z}_i$ for the i-th logical qubit satisfy the usual qubit algebra on $\mathcal{C}$.
- An element of $N(S) \setminus S$ is an **undetectable error**: it passes the syndrome check but changes the logical information. This is why it matters that the *code distance* $d = \min_{P \in N(S)\setminus S} \mathrm{wt}(P)$ is large.

6.4 Syndrome Measurement in the Stabilizer Formalism

Setup. For each generator g_i, prepare an ancilla qubit in $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, apply a controlled-$g_i$ (with ancilla as control), then measure the ancilla in the X basis.

The measurement outcome $s_i \in \{+1, -1\}$ (equivalently $s_i \in \{0, 1\}$) satisfies:

$$s_i = \begin{cases} +1 & \text{if the error } E \text{ commutes with } g_i \\ -1 & \text{if the error } E \text{ anticommutes with } g_i \end{cases}$$

The full collection $(s_1, \ldots, s_{n-k})$ is the **error syndrome**.

Worked Example 6.2: Complete syndrome calculation: Steane code, X_4 error

Consider the Steane code with generators as in Section 5. Suppose an X-error occurs on qubit 4.
We check each Z-type generator (they detect X-errors):

$$\begin{aligned} g_4 = Z_1Z_4Z_6Z_7: &\quad Z_4 \text{ anticommutes with } X_4 \Rightarrow s_4 = -1. \\ g_5 = Z_2Z_4Z_5Z_7: &\quad Z_4 \text{ anticommutes with } X_4 \Rightarrow s_5 = -1. \\ g_6 = Z_3Z_5Z_6Z_7: &\quad Z_3, Z_5, Z_6, Z_7 \text{ all commute with } X_4 \Rightarrow s_6 = +1. \end{aligned}$$

All X-type generators commute with X_4 (same Pauli type): $s_1 = s_2 = s_3 = +1$.
Syndrome:

$$(s_1, s_2, s_3, s_4, s_5, s_6) = (+1, +1, +1, -1, -1, +1).$$

The Z-syndrome in binary: $(s_4 - 1)/(-2), (s_5 - 1)/(-2), (s_6 - 1)/(-2) = (1, 1, 0) \to$ row pattern matching column 4 of H. This correctly identifies qubit 4. Apply X_4 to correct.

6.5 The Knill–Laflamme Error Correction Conditions

Theorem 6.1 (Knill–Laflamme conditions)**.** *A code with code projector $\Pi = \sum_i |c_i\rangle\langle c_i|$ (sum over an orthonormal basis $\{|c_i\rangle\}$ for $\mathcal{C}$) can correct a set of errors $\{E_a\}$ if and only if:*

$$\langle c_i | E_a^\dagger E_b | c_j \rangle = \delta_{ij} \lambda_{ab}$$

for all i, j and all errors E_a, E_b in the correctable set. Equivalently: $\Pi E_a^\dagger E_b \Pi = \lambda_{ab} \Pi$.

- **Non-degenerate condition ($i \neq j$):** Different codewords lead to orthogonal error spaces. The syndrome measurement can distinguish them.
- **Degenerate condition ($i = j$):** The "size" of each error looks the same from inside the code space. This means no encoded information is leaked.
- **For stabilizer codes:** The conditions reduce to requiring that $E_a^\dagger E_b$ either lies in S (trivial, correctable) or has non-trivial syndrome (detectable).

Worked Example 6.3: KL conditions for the 3-qubit bit-flip code

The code space basis is $\{|0\rangle_L, |1\rangle_L\} = \{|000\rangle, |111\rangle\}$. Consider errors $\{I, X_1, X_2, X_3\}$. Check the KL condition for $E_a = X_1$, $E_b = X_2$:

$$E_a^\dagger E_b = X_1 X_2.$$

$$\begin{aligned}
\langle 000| X_1X_2 |000\rangle &= \langle 000|110\rangle = 0, \\
\langle 111| X_1X_2 |111\rangle &= \langle 111|001\rangle = 0, \\
\langle 000| X_1X_2 |111\rangle &= \langle 000|001\rangle = 0, \\
\langle 111| X_1X_2 |000\rangle &= \langle 111|110\rangle = 0.
\end{aligned}$$

So $\Pi X_1 X_2 \Pi = 0$, which satisfies the KL condition with $\lambda_{12} = 0$. (The two errors lead to orthogonal error spaces.)

6.6 Graphical Summary: Stabilizer Code Structure

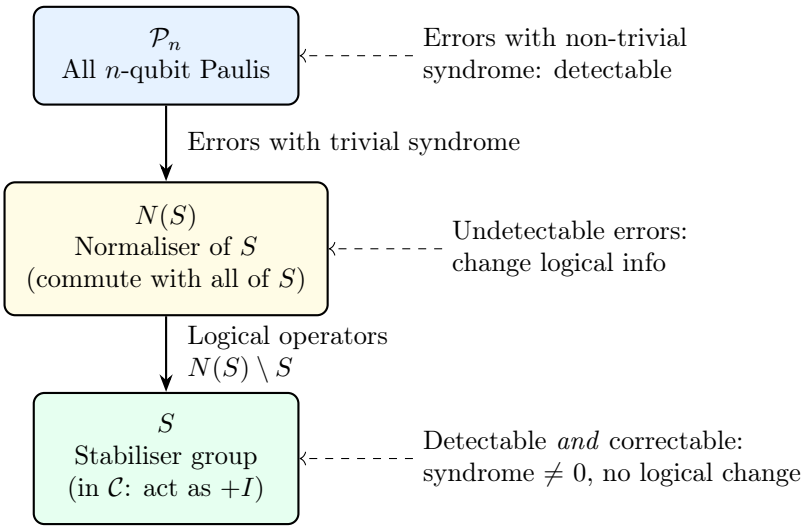


Figure 10: Hierarchy of the Pauli group relative to the stabiliser S. Errors in $\mathcal{P}_n \setminus N(S)$ are detectable; errors in $N(S) \setminus S$ are logical operators (undetectable and harmful); errors in S act trivially.

Exercise 6.1. For the 3-qubit bit-flip code, show that $X_1X_2X_3 \in N(S) \setminus S$ and that it acts as the logical $\bar{X}$ operator.

Exercise 6.2. Verify the KL conditions for the 3-qubit bit-flip code for the pair $E_a = X_1$, $E_b = X_1$ (the "diagonal" case $a = b$).

Exercise 6.3. Show that the Steane code satisfies the KL conditions for single-qubit Z-errors by checking that $\Pi Z_i^\dagger Z_j \Pi = 0$ for $i \neq j$ and $\Pi Z_i^2 \Pi = \Pi$ for any qubit i.

Section Summary

Section 6 key points:

- A stabilizer code is defined by an abelian Pauli subgroup S; the code space is the $+1$ eigenspace of all generators.

- n physical qubits, $n-k$ generators $\Rightarrow$ k logical qubits.
- Syndrome measurement projects errors onto Pauli components without disturbing logical information.
- KL conditions: a code corrects error set $\{E_a\}$ iff projected error products are proportional to the code projector.
- Undetectable errors $(N(S) \setminus S)$ act as logical operators; their minimum weight is the code distance d.

7 The Surface Code

Intuition

The surface code is the most promising candidate for practical quantum computers. Imagine a checkerboard. Data qubits sit on the *edges* of the board (the lines between squares). Two kinds of stabilizers sit on the squares themselves: "plaquette" stabilizers (on dark squares) measure Z parities and detect X errors; "vertex" stabilizers (on light squares) measure X parities and detect Z errors. An error creates a pair of "defects" on adjacent squares — like flipping two squares on the checkerboard — and correcting the error means pairing up defects and erasing them.

7.1 Lattice Structure

Consider a $d \times d$ square lattice where d is the **code distance**:

- **Data qubits** sit on the edges of the lattice (horizontal and vertical lines between vertices).
- **Z-stabilizers (plaquettes):** Each interior face of the lattice has a 4-qubit Z stabilizer $Z_{e_1} Z_{e_2} Z_{e_3} Z_{e_4}$, where $e_1, \ldots, e_4$ are the four edges bounding that face.
- **X-stabilizers (vertices):** Each interior vertex has a 4-qubit X stabilizer $X_{e_1} X_{e_2} X_{e_3} X_{e_4}$, where the edges are the four incident edges.
- Boundary qubits participate in 2- or 3-qubit stabilizers.

7.1.1 Counting for a $d = 3$ Surface Code

For $d = 3$, the lattice has $d^2 + (d-1)^2 = 9 + 4 = 13$ data qubits.

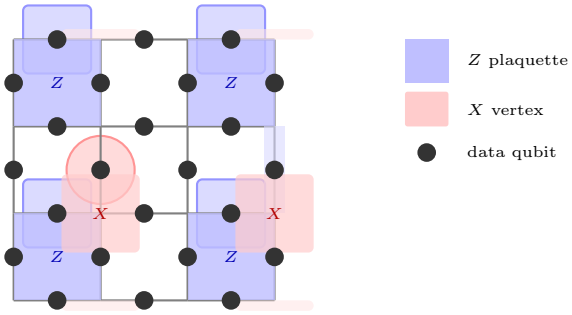


Figure 11: A $d = 3$ surface code lattice (schematic). Black dots are data qubits on edges. Blue squares are Z-stabilizer plaquettes; red regions are X-stabilizer vertices. This encodes 1 logical qubit in 13 data qubits with distance 3.

7.1.2 Code Parameters

For distance d:

$$\text{Data qubits: } n = d^2 + (d-1)^2 = 2d^2 - 2d + 1 \tag{10}$$

$$\text{Z-stabilizers: } (d-1)^2 + (d-1) = d(d-1) \tag{11}$$

$$\text{X-stabilizers: } d(d-1) \tag{12}$$

$$\text{Logical qubits: } k = 1 \quad \text{(for planar surface code)} \tag{13}$$

$$\text{Code distance: } d \quad \text{(minimum weight logical operator)} \tag{14}$$

Worked Example 7.1: Parameters for $d = 5$

$$n = 2(25) - 10 + 1 = 41 \text{ data qubits.}$$

Number of Z-stabilizers: $5 \times 4 = 20$.
Number of X-stabilizers: $5 \times 4 = 20$.
Total stabilizer measurements: $20 + 20 = 40$.
Logical qubits: $41 - 40 = 1$.
Code distance: $d = 5$, so we can correct up to $\lfloor (5-1)/2 \rfloor = 2$ errors.

7.2 Error Correction: Minimum-Weight Perfect Matching

How errors create syndromes. A chain of X-errors on a sequence of data qubits creates *defects* (syndrome -1 outcomes) at the *endpoints* of the chain.

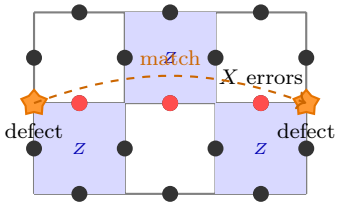


Figure 12: A chain of X-errors (red dots) on three data qubits creates *defects* (orange stars) only at the endpoints. The MWPM algorithm pairs up defects and infers the most likely error chain.

Minimum-weight perfect matching (MWPM).

1. Measure all stabilizers; collect the positions of defects (-1 outcomes).
2. Defects come in pairs (by the topology of the lattice).
3. Find the **minimum-weight perfect matching** — pair up defects so that the total number of data qubits inferred to have errors is minimised (using the Blossom algorithm or similar).
4. Apply the inferred correction.

7.3 Fault-Tolerance Threshold

Theorem 7.1 (Surface code threshold)**.** *For the surface code under depolarising noise with error rate p per gate:*

$$p_L \propto \left(\frac{p}{p_{\text{th}}} \right)^{\lfloor (d+1)/2 \rfloor},$$

*where $p_{\text{th}} \approx 1\%$ for standard depolarising noise. The logical error rate decreases **exponentially** with code distance d provided $p < p_{\text{th}}$.*

Worked Example 7.2: Logical error rate for $d = 5$

Take $p = 0.005$ (0.5%) and $p_{\text{th}} = 0.01$ (1%). Then $p/p_{\text{th}} = 0.5$ and:

$$p_L \propto (0.5)^{\lfloor 6/2 \rfloor} = (0.5)^3 = 0.125.$$

For $d = 7$: $p_L \propto (0.5)^4 = 0.0625$ — a factor 2 improvement per distance step. For $d = 11$:

$(0.5)^6 \approx 0.016$. This shows why large distance (and thus many physical qubits) are needed to reach extremely low logical error rates.

7.4 Logical Gates via Lattice Surgery

Direct transversal gates on the surface code are limited. Logical operations are instead performed by **lattice surgery**:

Logical CNOT: Merge two code patches (measuring joint stabilizers along the interface), perform joint measurements, then split.

Logical Hadamard: On the *rotated* surface code (a 45°-rotated lattice), $H^{\otimes n}$ implements $\bar{H}$.

Logical T gate: Requires magic state distillation (Section 8.4).

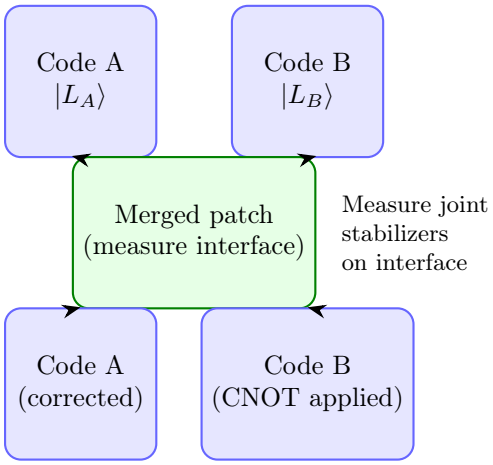


Figure 13: Lattice surgery for a logical CNOT. Two code patches are temporarily merged by measuring joint stabilizers on the interface, then split again. The net effect is a logical CNOT between qubits A and B.

7.5 Why the Surface Code is Practical

- **Local interactions only:** Each stabilizer involves at most 4 nearest-neighbour qubits — perfectly suited to 2D chip architectures (superconducting qubits, trapped ions arranged in a plane).
- **High threshold:** $p_{\text{th}} \approx 1\%$ is achievable with current hardware ($p \sim 0.1$–0.5%).
- **Simple decoding:** MWPM is efficient (polynomial time).
- **Natural fault-tolerance:** Stabilizer measurement errors can be tracked across repeated measurement rounds.

Exercise 7.1. Calculate the number of data qubits, Z-stabilizers, X-stabilizers, and logical qubits for a $d = 7$ surface code. Verify $k = 1$.

Exercise 7.2. A chain of two X-errors occurs on adjacent data qubits in a $d = 3$ surface code. Draw the defect pattern and explain how MWPM would (correctly) decode this error. Now suppose three consecutive X-errors form a chain spanning the lattice. Why does this cause a logical error?

Exercise 7.3. Below the threshold, the logical error rate scales as $p_L \propto (p/p_{\text{th}})^{(d+1)/2}$. How large must d be to achieve $p_L < 10^{-10}$ if $p = 0.003$ and $p_{\text{th}} = 0.01$?

Section Summary

Section 7 key points:

- The surface code places data qubits on lattice edges; Z (plaquette) and X (vertex) stabilizers detect bit- and phase-flip errors.
- A $d = d$ surface code uses $2d^2 - 2d + 1$ qubits, has distance d, and encodes 1 logical qubit.
- Errors create defect pairs; MWPM corrects by pairing defects optimally.
- Threshold $p_{\text{th}} \approx 1\%$; logical error rate drops exponentially with d below threshold.
- Logical gates via lattice surgery; T gate still requires magic state distillation.

8 Fault-Tolerant Quantum Gates

Intuition

Encoding qubits protects stored information, but we also need to *compute* on it. A gate performed directly on physical qubits might spread a single error into many errors — too many for the code to correct. Fault-tolerant gate design is the art of performing logical operations in a way that guarantees a single physical fault produces at most one logical fault.

8.1 Why Naive Gate Application Fails

Consider applying a transversal CNOT to the 3-qubit bit-flip code. A single fault in the gate could spread to affect multiple qubits. Suppose the second CNOT gate has an X error on its control:

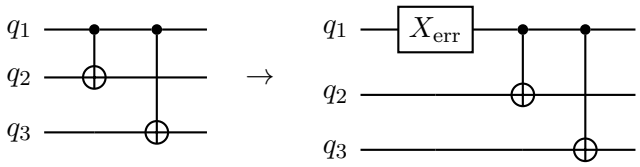


Figure 14: A single error on the first gate in the encoding circuit propagates to affect all subsequent qubits. This "error spreading" is the key danger that fault-tolerant design must prevent.

8.2 Transversal Gates

Definition 8.1 (Transversal gate)**.** A gate U_L is **transversal** if it can be written as $U_L = u^{\otimes n}$ (same single-qubit operation on every physical qubit, possibly with qubit permutations between code blocks).

A transversal gate is automatically fault-tolerant: a fault on physical qubit j affects only qubit j in the encoded block — it cannot spread to other qubits.

Code	Transversal gates	Missing gates
3-qubit bit-flip	$\overline{X}$, $\overline{Z}$	Phase, T, Hadamard
Steane $[\![7,1,3]\!]$	$\bar{H}$, $\bar{S}$, $\overline{\text{CNOT}}$ (Clifford group)	T gate
Surface code	$\bar{H}$ (rotated), $\overline{\text{CNOT}}$ via surgery	T gate

Theorem 8.1 (Eastin–Knill theorem)**.** *No quantum error-correcting code can have a transversal implementation of a* ***universal*** *gate set.*

This means every code must use some non-transversal technique for at least one gate. The T gate ($\pi/8$ gate) is the standard choice.

8.3 Fault-Tolerant Syndrome Measurement

Measuring a multi-qubit stabilizer like $Z_1Z_2Z_3Z_4$ requires ancilla qubits. A naive circuit might look like:

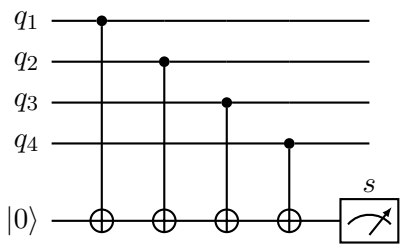


Figure 15: Standard ancilla-based syndrome measurement for the $Z_1Z_2Z_3Z_4$ stabilizer. The ancilla records the parity of the four data qubits. One fault in this circuit can propagate to multiple data qubits.

Caution

A single fault in the ancilla preparation or the CNOT gates can spread to multiple data qubits. For example, an X error on the ancilla before any CNOT will propagate a Z error to *every* data qubit it interacts with (since Z errors on the target propagate to the control). This can defeat the code's error-correcting power. The solution is to use **flag qubits** or **cat states** to detect such spreading.

8.3.1 Repeated Measurement

Even with perfect gates, measurement outcomes themselves are noisy. The standard approach is to repeat stabilizer measurements (typically 3 times for a distance-3 code) and use the majority outcome or track the syndrome history. For the surface code, syndrome measurements are repeated $O(d)$ times to reliably decode.

8.4 Magic State Distillation

8.4.1 Why We Need Magic States

We need a universal gate set. The Clifford gates (which include H, S, CNOT) are transversal in many codes, but they are *not* universal: any circuit built from Clifford gates alone can be simulated classically in polynomial time (Gottesman–Knill theorem).

To achieve universality, we add the T gate:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}.$$

The Clifford group plus T is universal. However, T is not transversal in standard codes.

8.4.2 The Magic State Idea

The T gate can be implemented by **gate teleportation** if we have access to the **magic state**:

$$|T\rangle = T\,|+\rangle = \frac{|0\rangle + e^{i\pi/4}\,|1\rangle}{\sqrt{2}}.$$

The teleportation circuit uses only Clifford gates plus a measurement:

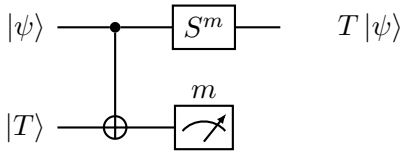


Figure 16: Gate teleportation implementing the T gate. Given one copy of the magic state $|T\rangle$, a Clifford correction S^m (where $m \in \{0, 1\}$ is the measurement result) yields $T|\psi\rangle$ using only Clifford gates.

8.4.3 Distillation Protocol

Physical T gates are noisy; the resulting states are close to (but not exactly) $|T\rangle$. The **15-to-1 distillation protocol** (Bravyi–Kitaev) takes 15 noisy magic states and produces 1 high-fidelity magic state:

- Input: 15 noisy $|T\rangle$ states, each with error probability ϵ.
- Output: 1 magic state with error probability $\approx 35\epsilon^3$ (superquadratic suppression).
- Circuit: uses only Clifford gates (which are fault-tolerant transversally) to distil the high-quality state.

Worked Example 8.1: Distillation improvement

If the physical T gate has error rate $\epsilon = 0.01$:

$$\text{After 1 round: } \epsilon_1 \approx 35(0.01)^3 = 35 \times 10^{-6} = 3.5 \times 10^{-5}.$$
$$\text{After 2 rounds: } \epsilon_2 \approx 35(3.5 \times 10^{-5})^3 \approx 1.5 \times 10^{-12}.$$

Two rounds of 15-to-1 distillation reduce the error from 1% to below 10^{-12} at the cost of $15^2 = 225$ initial magic states.

8.5 The Threshold Theorem

Theorem 8.2 (Fault-Tolerant Threshold Theorem)**.** *Suppose each physical gate, preparation, and measurement has error rate p, and errors are local (independent across qubits). There exists a threshold $p_{\text{th}} > 0$ such that: if $p < p_{\text{th}}$, arbitrarily long quantum computations can be performed with failure probability approaching 0. The qubit overhead scales as poly$(\log(1/\epsilon))$ for target error ϵ.*

- For the surface code: $p_{\text{th}} \approx 1\%$ (depolarising noise).
- For concatenated codes (Section 9): $p_{\text{th}} \approx 10^{-4}$ to 10^{-3}.
- The surface code's higher threshold is a major reason it is the leading practical candidate.

8.5.1 Below vs. Above Threshold

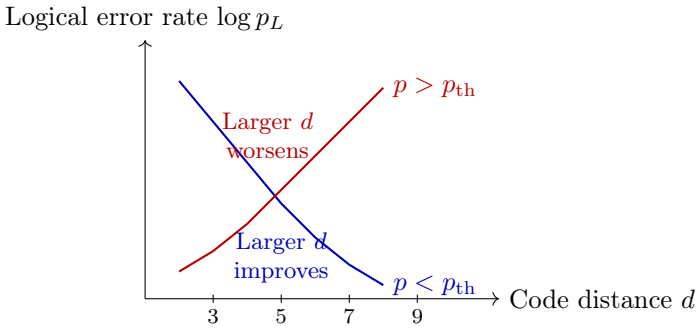


Figure 17: Below the threshold (blue), increasing code distance decreases the logical error rate exponentially. Above the threshold (red), adding more qubits makes things worse because there are too many opportunities for error.

Exercise 8.1. For a $d = 7$ surface code with $p = 0.004$ and $p_{\text{th}} = 0.01$, estimate the logical error rate as a multiple of $(p/p_{\text{th}})^{\lfloor (d+1)/2 \rfloor}$.

Exercise 8.2. Using the gate teleportation circuit of Figure 16, verify that measuring the ancilla in state $|1\rangle$ (outcome $m = 1$) and then applying S to the data qubit produces $T|\psi\rangle$. (*Hint:* work through the action of the CNOT on $|\psi\rangle|T\rangle$, then project onto $m = 0$ and $m = 1$ separately.)

Exercise 8.3. After three levels of 15-to-1 distillation starting from $\epsilon = 0.005$, what is the estimated output magic state error rate? How many physical T-gate preparations does this require?

Section Summary

Section 8 key points:

- Transversal gates prevent error spreading; the Eastin–Knill theorem says no code has a universal transversal gate set.
- Syndrome measurement circuits must be designed carefully to avoid fault propagation; repeated measurement is needed.
- The T gate is implemented via gate teleportation using magic states.
- Magic state distillation (15-to-1 protocol) converts many noisy $|T\rangle$ states into one high-fidelity one.
- The threshold theorem guarantees scalable quantum computation when physical error rates are below p_{th}.

9 Advanced Topics in Quantum Error Correction

Intuition

The surface code is excellent but has one major weakness: it encodes only 1 logical qubit no matter how many physical qubits you use. For a practical quantum computer running a million-gate algorithm, you might need thousands of logical qubits, requiring millions of physical qubits. This section surveys alternative codes that offer better rates (more logical qubits per physical qubit), higher thresholds, or additional transversal gates.

9.1 Quantum Low-Density Parity-Check (LDPC) Codes

Classical LDPC codes are a major tool in classical communications (used in 5G, WiFi, etc.). Each parity-check involves few bits (**sparse**) and each bit appears in few checks — hence "low density".

Quantum LDPC codes generalise this to the stabilizer setting:

- Each stabilizer generator involves at most w qubits (for some constant w, e.g. $w = 6$).
- Each qubit appears in at most w generators.
- Result: sparse syndrome measurement circuits, fewer gates per round.

9.1.1 Recent Breakthrough: Good Quantum LDPC Codes

Code family	**Rate** k/n	**Distance** d	**Stabilizer weight**
Surface code	$\Theta(1/n)$	$\Theta(\sqrt{n})$	4
Hypergraph product	Constant	$\Theta(\sqrt{n})$	$O(1)$
Lifted product (Panteleev–Kalachev 2022)	Constant	$\Theta(n)$	$O(1)$
Fiber bundle codes	Constant	$\Theta(n^{3/5})$	$O(1)$

A code with *constant rate* and *linear distance* ($d = \Theta(n)$) is called a **good quantum LDPC code**. These were only proved to exist in 2022, representing a major theoretical advance.

9.1.2 Practical Challenges

Despite better parameters, quantum LDPC codes face challenges:

- Generators can involve non-local (long-range) qubit pairs — hard to implement in 2D architectures.
- Efficient decoders are still being developed.
- Threshold values and exact overheads are not yet as well understood as for the surface code.

9.2 Color Codes

Color codes are topological codes defined on 3-colorable lattices.

9.2.1 2D Color Code Construction

Consider a 2D lattice where every face can be colored red, green, or blue, and every vertex touches exactly one face of each color. Data qubits sit on the vertices.

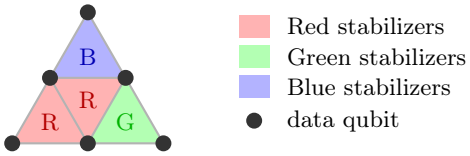


Figure 18: A small portion of a 2D color code lattice. Each colored face contributes both an X-type and a Z-type stabilizer on its vertices. The 3-colorability allows fault-tolerant gates beyond those of the surface code.

For each colored face f, there is an X-stabilizer and a Z-stabilizer on the vertices of f. The code is a $[\![n, 1, d]\!]$ code similar to the surface code.

9.2.2 Advantages of Color Codes

- The 2D color code has the same parameters as the surface code ($d = \Theta(\sqrt{n})$, rate $1/n$) but admits **transversal** H, S, CNOT, *and* T in some 3D variants.
- In 3D color codes, the full Clifford+T gate set is transversal, eliminating the need for magic state distillation — though at the cost of a 3D qubit architecture.
- Syndrome extraction is slightly more complex than for the surface code.

9.3 Concatenated Codes

Definition 9.1 (Code concatenation)**.** Given an "outer" code $C_1 = [\![n_1, k_1, d_1]\!]$ and an "inner" code $C_2 = [\![n_2, k_2, d_2]\!]$ with $k_2 = k_1$, the **concatenated code** $C_1 \circ C_2$ encodes each physical qubit of C_1 using the inner code C_2. The result is a $[\![n_1 n_2, k_1, d_1 d_2]\!]$ code.

Worked Example 9.1: Concatenating two Steane codes

Let both C_1 and C_2 be the Steane $[\![7, 1, 3]\!]$ code. The concatenated code $C_1 \circ C_2$ has:

$$n = 7 \times 7 = 49 \text{ physical qubits},$$
$$k = 1 \text{ logical qubit},$$
$$d = 3 \times 3 = 9.$$

After ℓ levels of concatenation:

$$n_\ell = 7^\ell, \quad d_\ell = 3^\ell, \quad k = 1.$$

Logical error rate under concatenation: Let p be the physical error rate. After ℓ levels:

$$p_L^{(\ell)} \approx \frac{1}{c}(cp)^{2^\ell},$$

where c is a constant depending on the code (for Steane, $c \approx 1/p_{\text{th}}$). For $p = 0.001$ and $c = 100$:

$$\ell = 1: \quad p_L \approx (0.1)^2 = 0.01.$$
$$\ell = 2: \quad p_L \approx (0.1)^4 = 10^{-4}.$$
$$\ell = 3: \quad p_L \approx (0.1)^8 = 10^{-8}.$$

Exponential suppression, but at the cost of $7^3 = 343$ physical qubits per logical qubit at level 3.

9.4 Comparison of QEC Code Families

Code	Rate	Distance	Transversal gates	Architecture
Surface	$\Theta(1/n)$	$\Theta(\sqrt{n})$	Clifford (partial)	2D local
Color (2D)	$\Theta(1/n)$	$\Theta(\sqrt{n})$	Full Clifford	2D local
Color (3D)	$\Theta(1/n)$	$\Theta(n^{1/3})$	Clifford + T	3D local
Quantum LDPC	Constant	up to $\Theta(n)$	Limited	Non-local
Concatenated	Variable	Exponential	Depends	Non-local

Exercise 9.1. After 4 levels of concatenation with a $[\![7, 1, 3]\!]$ code, how many physical qubits encode 1 logical qubit? What is the code distance?

Exercise 9.2. Explain why quantum LDPC codes with constant rate are important for practical quantum computation. What obstacles prevent their immediate use?

Exercise 9.3. A 2D color code and a surface code of the same physical qubit count n both have distance $\Theta(\sqrt{n})$. List two advantages of the color code and one advantage of the surface code.

Section Summary

Section 9 key points:

- Quantum LDPC codes offer constant rate (many logical qubits per physical qubit) with sparse stabilizers; recent breakthroughs achieve linear distance.
- Color codes are topological codes on 3-colorable lattices; 3D variants allow transversal T gates.
- Concatenated codes reduce logical error rates doubly-exponentially in the number of concatenation levels, at exponential qubit cost.
- No single code is optimal for all metrics; the choice depends on hardware architecture, target error rate, and gate set requirements.

10 Summary and Putting It All Together

This section brings together all the major themes of the course.

10.1 The Big Picture

Quantum error correction solves the fundamental problem of quantum computation: qubits decohere and gates are imperfect. The key ideas, in order of development, are:

1. **Errors are continuous, but correctable discretely.** Any single-qubit error decomposes into Pauli operators; syndrome measurement projects errors onto discrete Pauli faults.

2. **Entanglement protects information.** Encoding in entangled states spreads information holographically across many qubits. No individual qubit holds the secret.

3. **Syndromes are the key.** Measuring stabilizers reveals errors without collapsing the logical superposition. This is the central miracle of QEC.

4. **Fault-tolerance requires more than encoding.** Gates, measurements, and state preparations must all be performed in ways that prevent single faults from cascading.

5. **The threshold theorem gives hope.** Provided physical error rates are below $\sim 1\%$, reliable large-scale quantum computation is in principle possible.

10.2 Quick Reference: All Codes Covered

Code	n	k	d	**What it corrects**
3-qubit bit-flip	3	1	1	Single X error
3-qubit phase-flip	3	1	1	Single Z error
Shor 9-qubit	9	1	3	Any single-qubit error
Steane $[\![7, 1, 3]\!]$	7	1	3	Any single-qubit error
Surface code ($d = 5$)	41	1	5	Any 2-qubit error

10.3 Road Map Through the Material

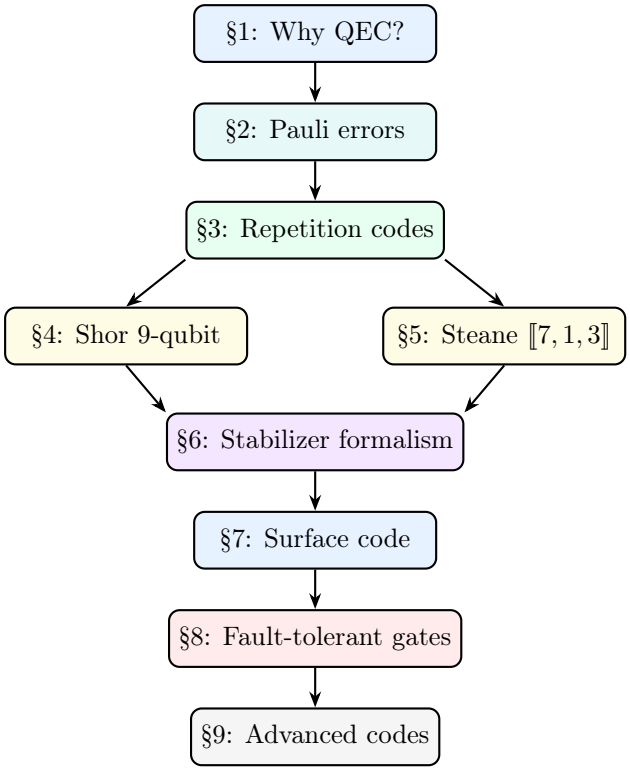


Figure 19: The logical dependency structure of these lecture notes. Each section builds on the previous; Sections 4 and 5 are parallel examples that Section 6 unifies.

10.4 Comprehensive Exercises

These exercises require combining knowledge from multiple sections.

Exercise 10.1. [Full worked pipeline.] A logical qubit is encoded in the 3-qubit bit-flip code. (a) Write the encoded state for $|\psi\rangle = \frac{1}{\sqrt{3}}|0\rangle + \sqrt{\frac{2}{3}}|1\rangle$. (b) Suppose a $Y_2 = iX_2Z_2$ error occurs. Compute the syndrome. (c) Apply the corrective operation and verify the state is restored.

Exercise 10.2. [KL conditions.] Prove that the Shor code satisfies the KL conditions for the error pair $E_a = Z_1$ (phase flip on qubit 1) and $E_b = Z_4$ (phase flip on qubit 4).

Exercise 10.3. [Code comparison.] For a quantum computer needing $p_L = 10^{-12}$ per logical gate with physical error rate $p = 0.001$: (a) How large must d be for the surface code? (b) How many levels of Steane code concatenation achieve this? (c) Which uses fewer physical qubits?

Exercise 10.4. [Steane code X-syndrome.] Suppose Z-errors occur on qubits 2 and 6 simultaneously in the Steane code. Compute the full X-syndrome and determine whether both errors can be identified and corrected. (*Hint:* Check whether the combined syndrome is the XOR of the individual syndromes.)

Exercise 10.5. [Surface code threshold.] A surface code operates at $p = 0.008$ with $p_{\text{th}} = 0.01$. At what code distance d does the logical error rate first drop below 10^{-6}? How many physical qubits does this require?

Exercise 10.6. [Magic state overhead.] A quantum algorithm requires 10^6 T-gate applications. Using the 15-to-1 distillation protocol with two rounds of distillation, and assuming each logical Clifford gate consumes one magic state, how many physical T-gate preparations are needed in total?

Exercise 10.7. [Error channel.] A qubit undergoes a dephasing channel with $p = 0.1$, followed by a bit-flip channel with $q = 0.05$. Write the combined error channel as a sum of Kraus operators and identify the probabilities of each Pauli error.

Exercise 10.8. [Stabilizer design.] Design a simple 5-qubit code (the "perfect code" $[\![5, 1, 3]\!]$) with stabilizers $\{XZZXI, IXZZX, XIXZZ, ZXIXZ\}$ (where we write operators on 5 qubits without tensor product symbols for brevity). (a) Verify that all four generators commute. (b) Show that the code encodes 1 logical qubit. (c) Show that it has distance 3.

Exercise 10.9. [Discretisation revisited.] A qubit suffers a general unitary error $U = \cos\theta\, I + i \sin\theta\, Z$ for $\theta = \pi/6$. The qubit is protected by the 3-qubit phase-flip code. (a) Decompose U in the Pauli basis. (b) After syndrome measurement, what is the probability that no correction is needed? That a correction is applied? (c) Verify that the final state is correct in both cases.

References

[1] Nielsen, M. A. & Chuang, I. L. (2010). *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press. *The standard reference. Chapters 10–11 cover QEC and fault tolerance.*

[2] Shor, P. W. (1995). Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4), R2493. *The original 9-qubit code paper.*

[3] Steane, A. M. (1996). Error correcting codes in quantum theory. *Physical Review Letters*, 77(5), 793. *The* $[\![7, 1, 3]\!]$ *CSS code.*

[4] Knill, E. & Laflamme, R. (1997). Theory of quantum error-correcting codes. *Physical Review A*, 55(2), 900. *The Knill–Laflamme error correction conditions.*

[5] Kitaev, A. Y. (1997). Quantum computations: algorithms and error correction. *Russian Mathematical Surveys*, 52(6), 1191. *The original surface/toric code paper.*

[6] Fowler, A. G., Mariantoni, M., Martinis, J. M., & Cleland, A. N. (2012). Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3), 032324. *Comprehensive surface code reference.*

[7] Bravyi, S. & Kitaev, A. (2005). Universal quantum computation with ideal Clifford gates and noisy ancillas. *Physical Review A*, 71(2), 022316. *The 15-to-1 magic state distillation protocol.*

[8] Panteleev, P. & Kalachev, G. (2022). Asymptotically good quantum and locally testable classical LDPC codes. *Proc. 54th ACM STOC*, 375–388. *Breakthrough result on good quantum LDPC codes.*

[9] Gottesman, D. (2009). *An introduction to quantum error correction and fault-tolerant quantum computation*. arXiv:0904.2557. *Excellent lecture notes covering stabilizer codes and fault tolerance. Freely available.*

Glossary of Key Terms

Ancilla qubit
: A helper qubit used to measure stabilizers without disturbing the data qubits. Prepared in a known state ($|0\rangle$ or $|+\rangle$), coupled to data, then measured.

Code distance d
: The minimum weight (number of non-identity Paulis) of any logical operator. A code of distance d corrects $\lfloor (d-1)/2 \rfloor$ errors.

CSS code
: A code constructed from two classical codes $C_1 \supseteq C_2$ with X-stabilizers from C_2 and Z-stabilizers from C_1. Guarantees X and Z generators commute.

Decoherence
: The loss of quantum coherence (superposition) due to entanglement with the environment. Characterised by times T_1 (energy relaxation) and T_2 (dephasing).

Fault-tolerant gate
: A gate implementation in which a single physical fault creates at most one logical fault.

Logical operator
: A Pauli operator that preserves the code space ($P \in N(S) \setminus S$) but acts non-trivially on the encoded qubit.

Magic state
: A non-stabilizer state $|T\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$ used via gate teleportation to implement the T gate.

No-cloning theorem
: It is impossible to copy an arbitrary unknown quantum state unitarily. Prevents direct repetition coding of qubits.

Stabilizer
: A Pauli operator g that fixes all codewords: $g\,|\psi\rangle = |\psi\rangle$. Equivalently, a $+1$ eigenstate condition.

Syndrome
: The pattern of ± 1 eigenvalues obtained by measuring all stabilizer generators. Uniquely identifies correctable errors.

Threshold p_{th}
: The maximum physical error rate below which increasing the code distance reduces the logical error rate. Approximately 1% for the surface code.

Transversal gate
: A logical gate implemented by applying the same single-qubit operation independently to each physical qubit. Automatically fault-tolerant.